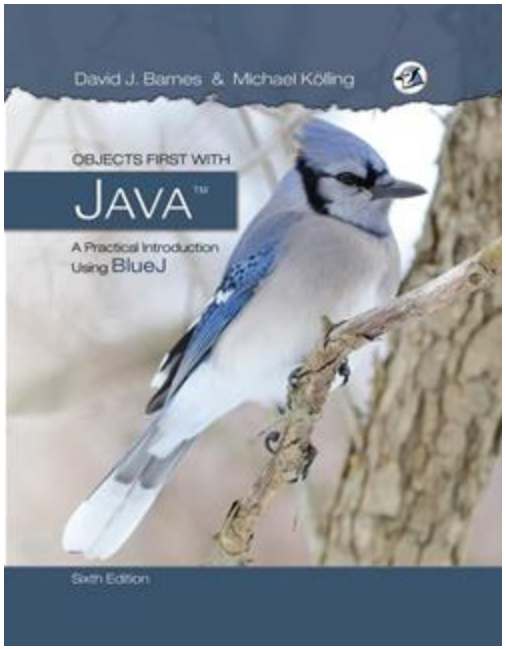
Unidad 2

Interacción de Objetos



Esta presentación es una traducción y adaptación de las presentaciones proporcionadas por la editorial Pearson de los capítulos 3, 4, 5 y 6 del libro “Objects First with Java: A Practical Introduction Using BlueJ, 6th Edition” escrito por David J. Barnes y Michael Kolling” © 2017

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Conceptos fundamentales

- modularización y abstracción
- tipos de datos: Primitivos y Objetos
- diagrama de clase
- creación de objetos
- diagrama de objetos
- sobrecarga de métodos
- interacción de objetos
- ocultando información
- Null
  - valores por defecto
- gestión interna de la memoria
  - tipos de datos primitivos
  - tipos de datos objeto
  - recolector de basura
  - tipos de datos primitivos – paso por valor
  - tipos de datos objeto – ¿paso por referencia?
- bibliotecas de Clases
- clase String
  - Identidad vs Igualdad
- documentación de clases
- uso de Javadoc para escribir documentación
- keyword this

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Conceptos fundamentales

- modularización y abstracción
- tipos de datos: Primitivos y Objetos
- diagrama de clase
- creación de objetos
- diagrama de objetos
- sobrecarga de métodos
- interacción de objetos
- ocultando información
- Null
  - valores por defecto
- gestión interna de la memoria
  - tipos de datos primitivos
  - tipos de datos objeto
  - recolector de basura
  - tipos de datos primitivos – paso por valor
  - tipos de datos objeto – ¿paso por referencia?
- bibliotecas de Clases
- clase String
  - Identidad vs Igualdad
- documentación de clases
- uso de Javadoc para escribir documentación
- keyword this

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Modularización y Abstracción



Modularización y abstracción son complementarios



Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

¿Cómo diseñar un coche?

- Los ingenieros aplican **Divide y Vencerás:**

– **Abstracción**:

habilidad de ignorar los detalles de las partes para centrarse en niveles superiores del problema.

- A nivel superior, ven cada parte como un componente sencillo que es combinado con otros módulos haciendo uso de sus **interfaces estándar** para construir componentes más complejos



Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Images Source: http://www.autoevolution.com/news/lexus-ct-200h-disassembled-after-100000km-german-endurance-test-79389.html

¿Cómo diseñar un coche?

- Los ingenieros aplican **Divide y Vencerás:**

– **Modularización**:

proceso de dividir un todo en partes bien definidas, que pueden ser construidas y revisadas de forma independiente y que interactúan de la forma especificada.

- A nivel inferior, dividen el coche en módulos independientes
- distinta gente trabaja en distintos módulos
- **¡¡tienen que acordar unos interfaces estándar para poder interactuar!!**

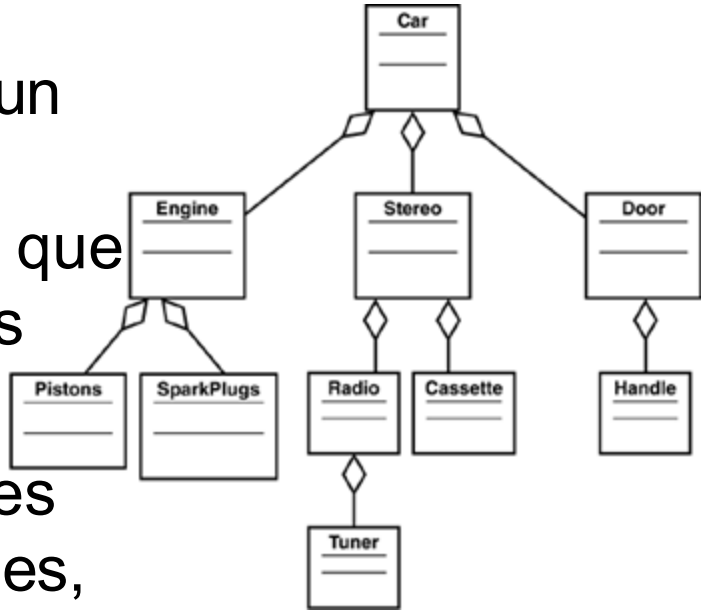


Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Images Source: http://www.autoevolution.com/news/lexus-ct-200h-disassembled-after-100000km-german-endurance-test-79389.html

Modularización y Abstracción

- **Los mismos principios se aplican en desarrollo software.**
  - Mantenemos una visión general de un programa complejo tratando de identificar subcomponentes (clases) que podemos programar como entidades independientes.
  - Luego usamos esos subcomponentes (clases) como si fueran partes simples, sin preocuparnos por sus complejidades internas.



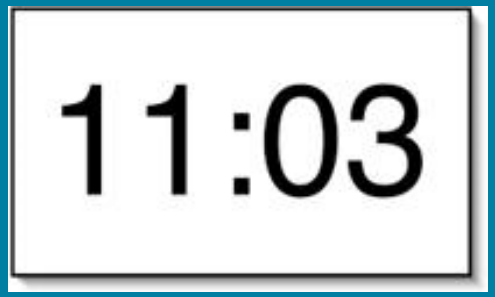
Identificar las clases que debería tener un sistema de software para cualquier problema dado no es fácil

Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Images Source: https://flylib.com/books/en/3.477.1.69/1/

Demo de Proyecto “clock-display” (Sample Code Unit 2)

Un reloj digital



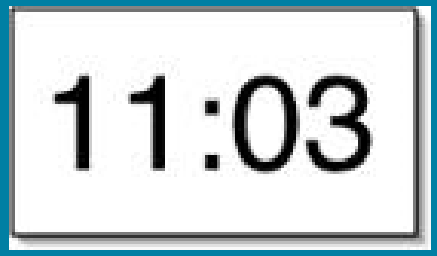
Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Demo de Proyecto “clock-display” (Sample Code Unit 2)

Modularizando el Reloj Digital

¿Un reloj con 1 pantalla de 4 dígitos?



¿O un reloj con 2 pantallas de 2 dígitos?



Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Demo de Proyecto “clock-display” (Sample Code Unit 2)

Modelando una pantalla de 2 dígitos

- Llamamos a la clase `NumberDisplay`.
- Dos campos de tipo entero:
  - El valor actual.
  - El límite para dicho valor.
- El valor actual se incrementa hasta que alcanza el límite.
  - Vuelve a 0 al alcanzar el límite.

Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Conceptos fundamentales

- modularización y abstracción
- tipos de datos: Primitivos y Objetos
- diagrama de clase
- creación de objetos
- diagrama de objetos
- sobrecarga de constructores
- sobrecarga de métodos
- interacción de objetos
- ocultando información
- Null
  - valores por defecto
- gestión interna de la memoria
  - tipos de datos primitivos
  - tipos de datos objeto
  - recolector de basura
  - tipos de datos primitivos – paso por valor
  - tipos de datos objeto – ¿paso por referencia?
- bibliotecas de Clases
- clase String
  - Identidad vs Igualdad
- documentación de clases
- uso de Javadoc para escribir documentación
- keyword this

2, 2009 Pearson Education, Inc. All Rights Reserved

Tipos de datos: Primitivos y Objetos

- Los datos pueden ser clasificados de distintos tipos;
  - p.ej. enteros, booleanos, punto flotante...
- Además, cada clase es un tipo de dato único
  - p.ej. `String`, `TicketMachine`, `NumberDisplay`.
- Por tanto los tipos de datos puede ser compuestos (Objetos) y no solo valores simples (Primitivos).

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Tipos de datos: Primitivos y Objetos

```
public class NumberDisplay
{
    private int limit;
    private int value;

    public NumberDisplay(int limit)
    {
        this.limit = limit;
        value = 0;
    }
    ...  //Se omiten constructores y métodos
}
```

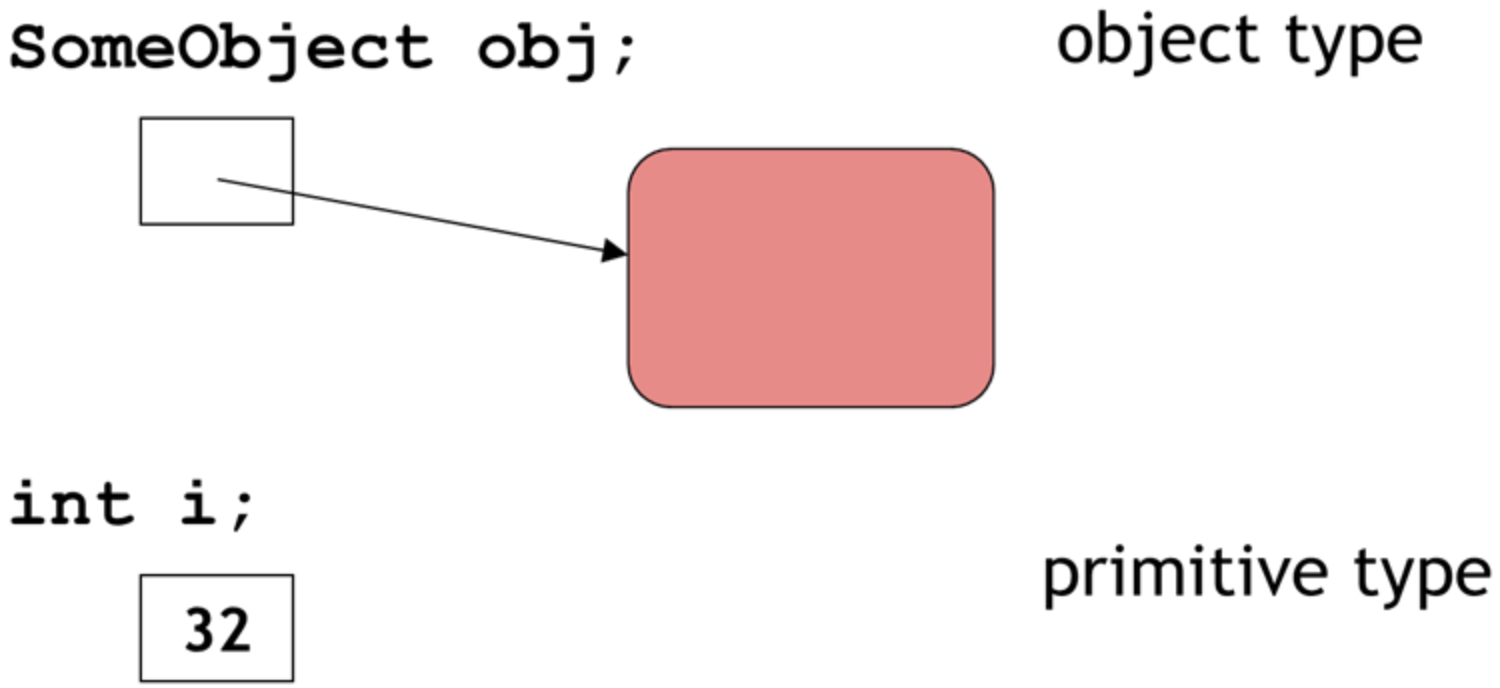
¡Las Clases definen tipos de datos!

```
public class ClockDisplay
{
        private NumberDisplay hours;
        private NumberDisplay minutes;
...     ////Se omiten constructores y métodos
}
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

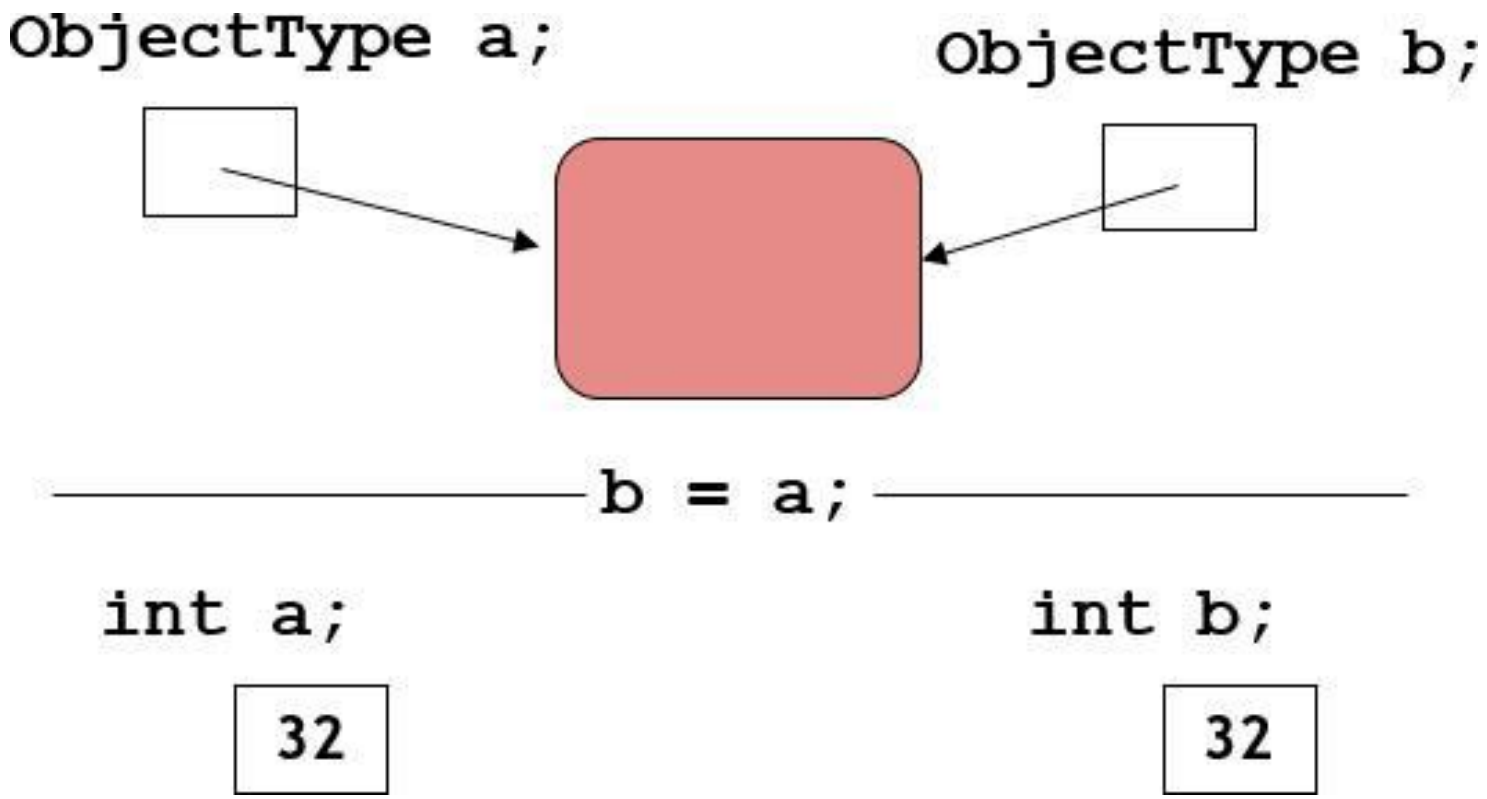
Tipos de datos: Primitivos y Objetos



Datos de tipo Primitivo

Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Tipos de datos: Primitivos y Objetos



Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Conceptos fundamentales

- modularización y abstracción
- tipos de datos: Primitivos y Objetos
- diagrama de clase
- creación de objetos
- diagrama de objetos
- sobrecarga de métodos
- interacción de objetos
- ocultando información
- Null
  - valores por defecto
- gestión interna de la memoria
  - tipos de datos primitivos
  - tipos de datos objeto
  - recolector de basura
  - tipos de datos primitivos – paso por valor
  - tipos de datos objeto – ¿paso por referencia?
- bibliotecas de Clases
- clase String
  - Identidad vs Igualdad
- documentación de clases
- uso de Javadoc para escribir documentación
- keyword this

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Diagrama de clase

Diagrama UML

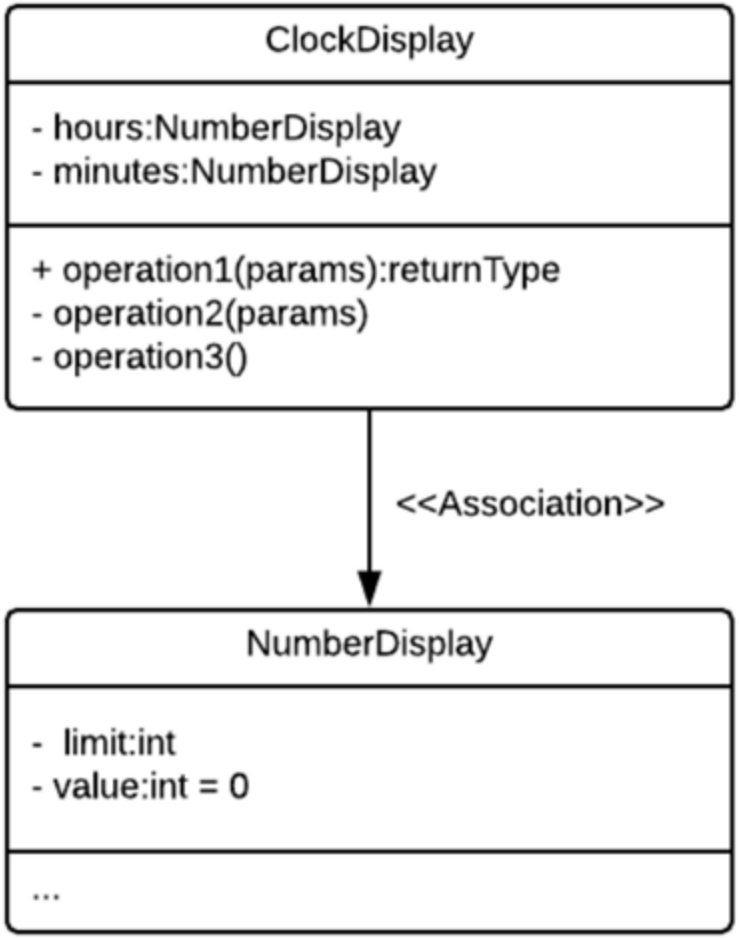




Diagrama BlueJ



Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Conceptos fundamentales

- modularización y abstracción
- tipos de datos: Primitivos y Objetos
- diagrama de clase
- creación de objetos
- diagrama de objetos
- sobrecarga de métodos
- interacción de objetos
- ocultando información
- Null
  - valores por defecto
- gestión interna de la memoria
  - tipos de datos primitivos
  - tipos de datos objeto
  - recolector de basura
  - tipos de datos primitivos – paso por valor
  - tipos de datos objeto – ¿paso por referencia?
- bibliotecas de Clases
- clase String
  - Identidad vs Igualdad
- documentación de clases
- uso de Javadoc para escribir documentación
- keyword this
- Anexos
  - depurador

Pearson

Copyright © 2017, 201

Creación de objetos

```
public class NumberDisplay
{
    private int limit;
    private int value;

    public NumberDisplay(int limit)
    {
        this.limit = limit;
        value = 0;
    }
    ... //Se omiten constructores y métodos
}

 public class ClockDisplay
 {
        private NumberDisplay hours;
        private NumberDisplay minutes;
 ...    //Constructor and methods omitted
 }
```

¡hours y minutes son variables de tipo objeto declaradas en ClockDisplay!

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Creación de objetos

¡hours y minutes son instanciadas como nuevos objetos dentro del constructor de ClockDisplay!

```
public class ClockDisplay
{
    private NumberDisplay hours;
    private NumberDisplay minutes;
    private String displayString;

    public ClockDisplay()
    {
        hours = new NumberDisplay(24);
        minutes = new NumberDisplay(60);
        …
    }
}
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

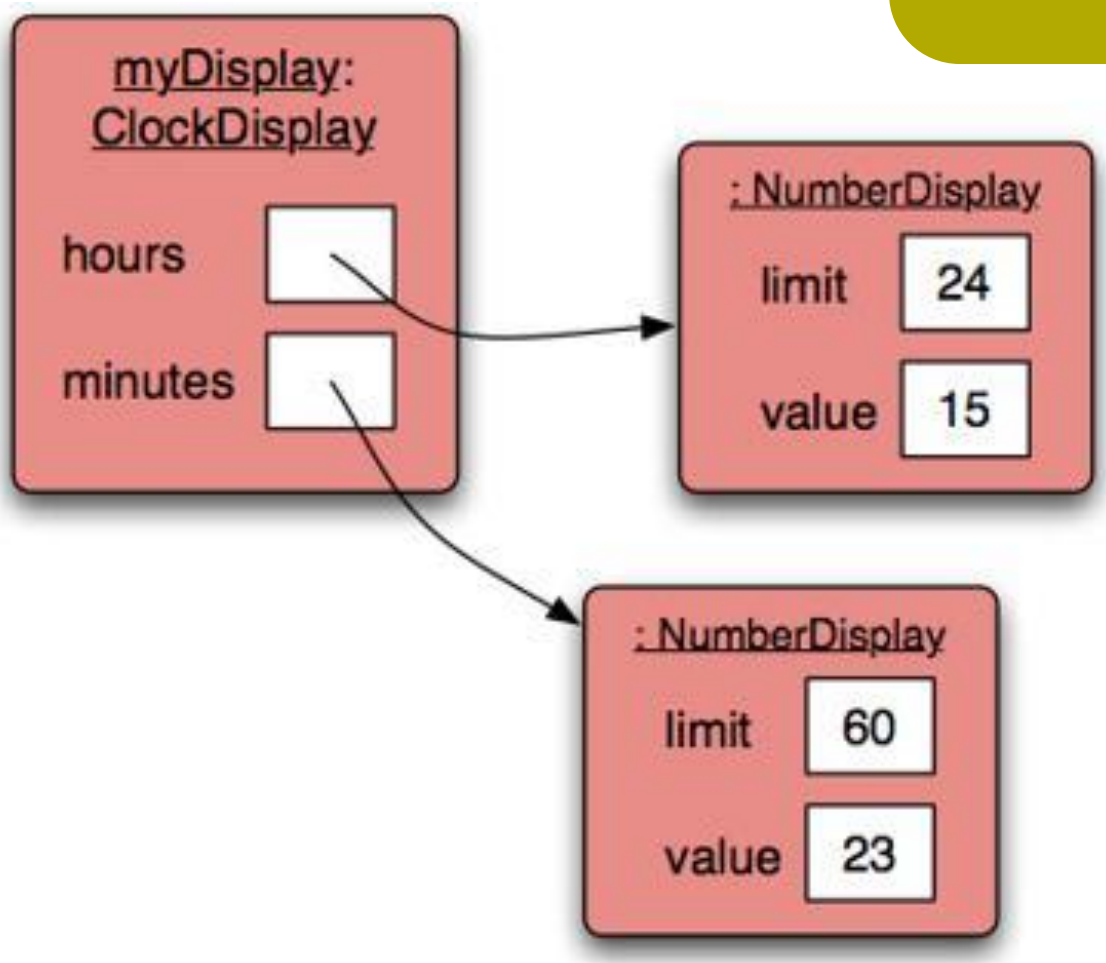
Conceptos fundamentales

- modularización y abstracción
- tipos de datos: Primitivos y Objetos
- diagrama de clase
- creación de objetos
- diagrama de objetos
- sobrecarga de constructores
- sobrecarga de métodos
- interacción de objetos
- ocultando información
- Null
  - valores por defecto
- gestión interna de la memoria
  - tipos de datos primitivos
  - tipos de datos objeto
  - recolector de basura
  - tipos de datos primitivos – paso por valor
  - tipos de datos objeto – ¿paso por referencia?
- bibliotecas de Clases
- clase String
  - Identidad vs Igualdad
- documentación de clases
- uso de Javadoc para escribir documentación
- keyword this

2, 2009 Pearson Education, Inc. All Rights Reserved

Diagrama de Objetos

Las variables de tipo objeto almacenan **referencias** a objetos



Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Conceptos fundamentales

- modularización y abstracción
- tipos de datos: Primitivos y Objetos
- diagrama de clase
- creación de objetos
- diagrama de objetos
- sobrecarga de métodos
- interacción de objetos
- ocultando información
- Null
  - valores por defecto
- gestión interna de la memoria
  - tipos de datos primitivos
  - tipos de datos objeto
  - recolector de basura
  - tipos de datos primitivos – paso por valor
  - tipos de datos objeto – ¿paso por referencia?
- bibliotecas de Clases
- clase String
  - Identidad vs Igualdad
- documentación de clases
- uso de Javadoc para escribir documentación
- keyword this

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Sobrecarga de Métodos (Constructores)

Ejemplo (código Fuente de la clase "ClockDisplay")

```
public class ClockDisplay  {
      …
        public ClockDisplay ( ){
        //code omitted
        }

        public ClockDisplay (int hour, int minute){
        //code omitted
        }
        …
}
```

¡¡Sobrecarga!!

Diferente número de parámetros

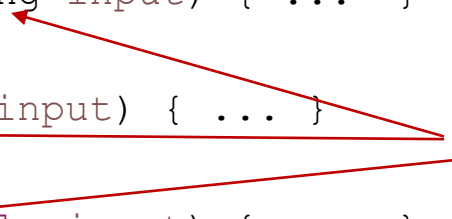
Sobrecargar (**overload**) un constructor es definir varios constructores que solo se diferencian en el número y/o tipo de parámetros que admiten

Pearson Reserved

Sobrecarga de Métodos

El mismo principio también funciona con los métodos

```
1  void println(String input) { ... }
2
3  void println(int input) { ... }
4
5  void println(double input) { ... }
```



¡¡Sobrecarga!!

Diferentes tipos de parámetros

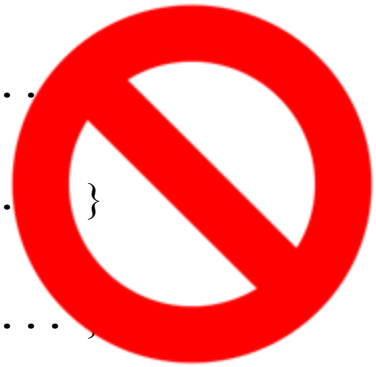
Sobrecargar (**overload**) un método es definir varios métodos que solo se diferencian en el número y/o tipo de parámetros que admiten

Pearson Reserved

Sobrecarga de Métodos

No es suficiente con tener distintos tipos de retorno

```
1 double add(int a, int b) { ..
2
3    int add(int a, int b) { ... }
4
5 String add(int a, int b) { ... }
```



¡¡No Sobrecarga!!

Sobrecargar (**overload**) un método es definir varios métodos que solo se diferencian en el número y/o tipo de parámetros que admiten

Pearson Reserved

Sobrecarga de Métodos

No es suficiente con tener distintos nombres de parámetros

```
1     int add(int a, int b) {    }
2
3     int add(int x, int y) {  . }
4
5
```



¡¡No Sobrecarga!!

Sobrecargar (**overload**) un método es definir varios métodos que solo se diferencian en el número y/o tipo de parámetros que admiten

Pearson Reserved

Conceptos fundamentales

- modularización y abstracción
- tipos de datos: Primitivos y Objetos
- diagrama de clase
- creación de objetos
- diagrama de objetos
- sobrecarga de métodos
- interacción de objetos
- ocultando información
- Null
  - valores por defecto
- gestión interna de la memoria
  - tipos de datos primitivos
  - tipos de datos objeto
  - recolector de basura
  - tipos de datos primitivos – paso por valor
  - tipos de datos objeto – ¿paso por referencia?
- bibliotecas de Clases
- clase String
  - Identidad vs Igualdad
- documentación de clases
- uso de Javadoc para escribir documentación
- keyword this

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Interacción de objetos

- Dos objetos interactúan cuando **uno invoca** un **método del otro**.
- La **interacción** es **usualmente en una dirección**
  - **cliente-servidor**
  - El objeto **cliente** puede **pedir** al objeto **servidor** que **haga algo**
  - El objeto **cliente** puede **pedir** al objeto **servidor** que le **devuelva datos**

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Interacción de objetos

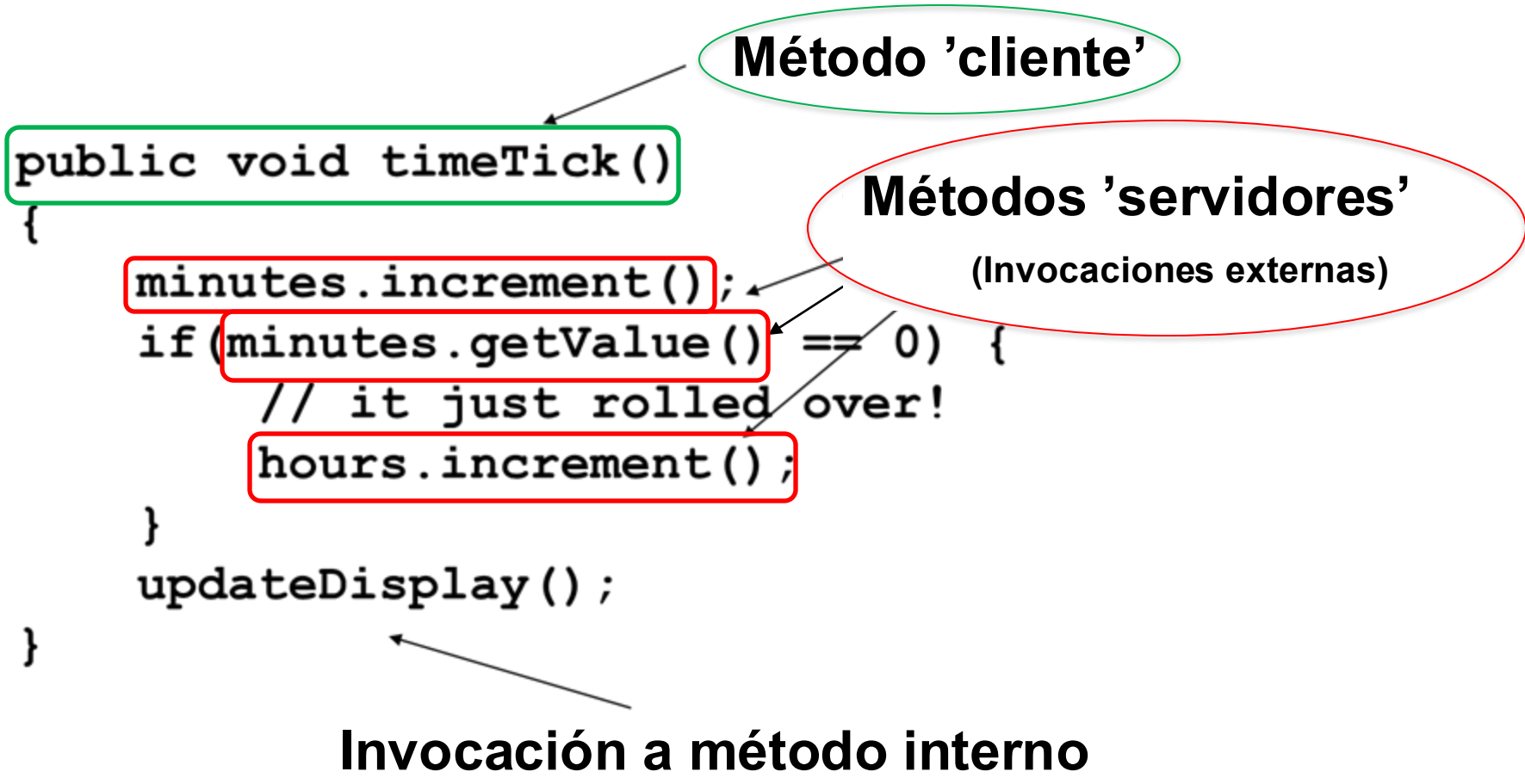
- 2 objetos NumberDisplay almacenan datos del objeto ClockDisplay.
  - **ClockDisplay** es el objeto '**cliente**'.
  - Los objetos **NumberDisplay** son objetos '**servidores**'.
  - El **cliente invoca métodos** en los objetos **servidores**.

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Interacción de objetos

Invocación de Métodos



Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Interacción de objetos

Invocación Externa de Métodos

- Forma general:
  objeto . nombreMétodo (parámetros)
- Ejemplos:
  `hours.increment()`
  `minutes.getValue()`

```
public class ClockDisplay{
        …
        public void timeTick() {
                minutes.increment();
                if(minutes.getValue() == 0) { // it just rolled over!
                    hours.increment();
                }
                updateDisplay();
        }
        …
}
```

¡Fijate en el estilo del código!

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Interacción de objetos

Invocación Interna de Métodos

- **No** es necesario indicar nombre de variable:
  `updateDisplay();`
- Si método es solo para uso interno tiene visibilidad **`private`**
  - Para **prevenir** su invocación desde fuera de la clase

```
public class ClockDisplay  {
      …
        public ClockDisplay ( ) {
                hours = new NumberDisplay(24);
                minutes = new NumberDisplay(60);
                updateDisplay();
        }
        …
        private void updateDisplay( ) {
                displayString = hours.getDisplayValue() + ":" +
                minutes.getDisplayValue();
        }
        …
}
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Conceptos fundamentales

- modularización y abstracción
- tipos de datos: Primitivos y Objetos
- diagrama de clase
- creación de objetos
- diagrama de objetos
- sobrecarga de métodos
- interacción de objetos
- ocultando información
- Null
  - valores por defecto
- gestión interna de la memoria
  - tipos de datos primitivos
  - tipos de datos objeto
  - recolector de basura
  - tipos de datos primitivos – paso por valor
  - tipos de datos objeto – ¿paso por referencia?
- bibliotecas de Clases
- clase String
  - Identidad vs Igualdad
- documentación de clases
- uso de Javadoc para escribir documentación
- keyword this

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Ocultando información

- El principio de ocultamiento de la información es un principio que afirma que los detalles internos de la implementación de una clase deberían estar ocultos para otras clases.
  - Es decir, los datos de un objecto están **ocultos** para otros objectos (***abstracción, encapsulación***).
- Sabemos **qué** puede hacer un objecto, no **cómo** lo hace.
- Ocultar información incrementa el nivel de **independencia** (***modularización***).
- La independencia de módulos es importante para el mantenimiento de grandes sistemas (**bajo acoplamiento (*loose coupling*)**).

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Ocultando información

Modificadores de acceso

- Controlan quién (qué clases) puede acceder a los elementos de nuestra clase
  - Dentro de una clase, todos sus elementos son accesibles
- Los elementos **public** son accesibles para objetos de otras clases:
  - Campos, constructores y métodos
- Los campos no deben ser **public**.
- Los elementos **private** son accesibles sólo para objetos de la misma clase.
- Sólo los métodos que van a ser accesibles desde otras clases deben ser públicos.

Ocultando información

Modificadores de acceso

```
1  public     class  Minion{
2     public  void   sayHello(){ //...
        };
3  }
```

```
1  public class  ExamplePublic{
2     public static void main(String[] args){
3          Minion stuart= new  Minion();
4          stuart.sayHello();
5     }
6  }
```

public

- Puede ser accedido desde cualquier clase
- Se usa `public` para clases, constructores y métodos

Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Ocultando información

Modificadores de acceso

```
1 public    class Minion{
2    private String name;
3 }
```

```
1 public    class  ExamplePrivate{
2    public static void   main(String[] args){
3         Minion stuart =  new Minion();
4         stuart.name      =      "Stuart";
5
6    }
   }
```

private

- No es accesible desde fuera de la clase (sólo accesible desde la misma clase)
- Se usa `private` para los campos o atributos

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Ocultando información

Modificadores de acceso

```
1  public    class  Minion{
2     private String name;
3      Minion(String name){
4           this.name=name;
5      }
6      public void greet(Minion other){
7           System.out.println("Hi " + other.name);
8      }
9
10 public class     ExamplePrivate2{
11     public static void main(String[] args) {
12          Minion stuart = new Minion("Stuart");
13.         Minion kevin = new Minion("Kevin");
14         stuart.greet(kevin);
15      }
16 }
```

private

- Los objetos de una clase pueden acceder a elementos privados de otros objetos de la misma clase

Ocultando información

Encapsulación

```
1  public     class  Minion{
2     String name;
3     int     age;
4     boolean isAlive;
5     String pinCode;
6  }
```



```
1  public     class  Minion {
2     private String name;
3     private int    age;
4     private boolean isAlive;
5     private String pinCode;
6  }
```



Encapsulación

- Evita accesos y modificaciones no deseados (o no controlados) de los campos

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Ocultando información

Recordatorio de métodos get y set

```
1  public       class  Minion{
2     private String name;
3
4     public String getName(){
5          return       this.name;
6     }
7     public void setName(String value){
8          this.name = value;
9     }
10 }
```

Métodos get

- Acceso de lectura

Métodos set

- Acceso de escritura

private

- Como convención: declarar los campos como `private`
- Acceder a los atributos `private` mediante métodos `public` *get* y *set* para controlar el acceso de lectura y escritura

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Ocultando información

Resumen de Modificadores de acceso

	Clase	*Package*	Otras clases
public (Accesible desde cualquier sitio)	accesible	accesible	accesible
protected (Accesible desde esa clase, sus subclases y todas las clases del mismo paquete)	accesible	accesible	no accesible
sin modificador (package-private) (Accesible desde esa clase y todas las clases del mismo paquete)	accesible	accesible	no accesible
private (Accesible sólo desde esa clase)	accesible	no accesible	no accesible

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Conceptos fundamentales

- modularización y abstracción
- tipos de datos: Primitivos y Objetos
- diagrama de clase
- creación de objetos
- diagrama de objetos
- sobrecarga de métodos
- interacción de objetos
- ocultando información
- Null
  - valores por defecto
- gestión interna de la memoria
  - tipos de datos primitivos
  - tipos de datos objeto
  - recolector de basura
  - tipos de datos primitivos – paso por valor
  - tipos de datos objeto – ¡paso po referencia?
- bibliotecas de Clases
- clase String
  - Identidad vs Igualdad
- documentación de clases
- uso de Javadoc para escribir documentación
- keyword this

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Null

- Se usa con tipos objeto para indicar que "no hay objeto".
- Todas las variables de tipo objeto de cualquier clase son inicialmente null.
- Se puede comprobar si una variable objeto tiene el valor null:

```
if(myObject == null);
```

- También se puede asignar null a variables
  - pierden la referencia que tuvieran.

```
myObject = null;
```

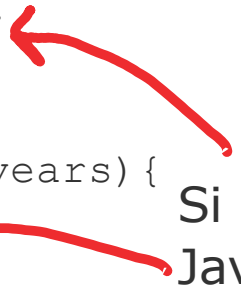
- OJO, si se intenta invocar un método de una variable de tipo objeto que se piensa que tiene un objeto pero en cambio contiene null: **`NullPointerException`**

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Null

```
1   class       Person {
2    public String name;
3    public int age;
4    public Person(int years){
5     age=years;
6    }
7    public String getName(){
8     return name;
9    }
10   public static void main(String[] args){
11    Person john = new Person(23);
12    System.out.println(john.getName());
13    System.out.println(john.getName().toLowerCase());
14   }
15}
```





Si un campo no se inicializa, Java fija su valor por defecto. En el caso de un String es `null`

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Null

```
1   class       Person {
2    public String name;
3    public int age;
4    public Person(int years){
5     age=years;
6    }
7    public String getName(){
8     return name;
9    }
10   public static void main(String[] args){
11    Person john = new Person(23);
12    System.out.println(john.getName());
13    System.out.println(john.getName().toLowerCase());
14   }
15}
```

Si un campo no se inicializa, Java fija su valor por defecto. En el caso de un String es `null`

Devuelve contenido de name: null

Problema: se invoca método de String pero en realidad contiene null

```
>_ null
   Exception occurred.
   java.lang.NullPointerException
           at Person.main(Person.java:13)
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Null

```
1  class      Person {
2   public String name;
3   public int age;
4   public Person(String firstName,int years){
5    name=firstName;
6    age=years;
7   }
8   public String getName(){
9    return name;
10   }
11  public static void main(String[] args){
12   Person john = new Person("John",23);
13   System.out.println(john.getName());
14   System.out.println(john.getName().toLowerCase());
15  }
16}
```

Solución 1: arreglar el problema

```
> John
  john
```

Pearson

Null

```
1   class       Person {
2    public String name;
3    public int age;
4    public Person(int years){
5     age=years;
6    }
7    public String getName(){
8     return name;
9    }
10   public static void main(String[] args){
11    Person john = new Person(23);
12    if (john.getName() != null){
13     System.out.println(john.getName().toLowerCase());
14    }
15   }
16}
```

Solución 2: si se espera que un cierto valor podría ser **`null`**

(p.ej. una entrada desde teclado).
Comprueba si es `null`



Pearson

ved

Conceptos fundamentales

- modularización y abstracción
- tipos de datos: Primitivos y Objetos
- diagrama de clase
- creación de objetos
- diagrama de objetos
- sobrecarga de métodos
- interacción de objetos
- ocultando información
- Null
  - valores por defecto
- gestión interna de la memoria
  - tipos de datos primitivos
  - tipos de datos objeto
  - recolector de basura
  - tipos de datos primitivos – paso por valor
  - tipos de datos objeto – ¿paso por referencia?
- bibliotecas de Clases
- clase String
  - Identidad vs Igualdad
- documentación de clases
- uso de Javadoc para escribir documentación
- keyword this

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Null

Valores por defecto

- Si el programador no inicializa los campos, Java les proporciona un valor por defecto
  - ¡¡Las variables locales no tienen valores por defecto!!



  - A las variables locales SIEMPRE hay que darles un valor antes del primer uso.

Tipo de dato	Valor por defecto en los campos
int	0
double	0.0
char	'\u0000'
boolean	false
String	null
Todos los Objetos	null

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Conceptos fundamentales

- modularización y abstracción
- tipos de datos: Primitivos y Objetos
- diagrama de clase
- creación de objetos
- diagrama de objetos
- sobrecarga de métodos
- interacción de objetos
- ocultando información
- Null
  - valores por defecto
- gestión interna de la memoria
  - tipos de datos primitivos
  - tipos de datos objeto
  - recolector de basura
  - tipos de datos primitivos – paso por valor
  - tipos de datos objeto – ¿paso por referencia?
- bibliotecas de Clases
- clase String
  - Identidad vs Igualdad
- documentación de clases
- uso de Javadoc para escribir documentación
- keyword this

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Gestión Interna de la Memoria

Tipos de datos Primitivos

Dirección	Identificador	Valor
#0		
#1	`maxSpeed`	
#2		
#3		
#4		
#5		

Declaración

```
int
        maxSpeed
```

- Se asigna la memoria
- La cantidad de memoria asignada depende del tipo de datos
- El identificador se usa para recuperar el valor de la memoria

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Gestión Interna de la Memoria

Tipos de datos Primitivos

Dirección	Identificador	Valor
#0		
#1	maxSpeed	42
#2		
#3		
#4		
#5		

Asignación

```
maxSpeed = 42;
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Conceptos fundamentales

- modularización y abstracción
- tipos de datos: Primitivos y Objetos
- diagrama de clase
- creación de objetos
- diagrama de objetos
- sobrecarga de métodos
- interacción de objetos
- ocultando información
- Null
  - valores por defecto
- gestión interna de la memoria
  - tipos de datos primitivos
  - tipos de datos objeto
  - recolector de basura
  - tipos de datos primitivos – paso por valor
  - tipos de datos objeto – ¿paso por referencia?
- bibliotecas de Clases
- clase String
  - Identidad vs Igualdad
- documentación de clases
- uso de Javadoc para escribir documentación
- keyword this

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Gestión Interna de la Memoria

Tipos de datos Objeto

Dirección	Identificador	Valor
#0		null
#1	hours	#0
#2		
#3		
#4		
#5		

Declaración

```
NumberDisplay hours;
```

- Referencia sin inicializar apuntando a `null`
- El "Valor" de los objetos es una referencia apuntando al objeto real

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Gestión Interna de la Memoria

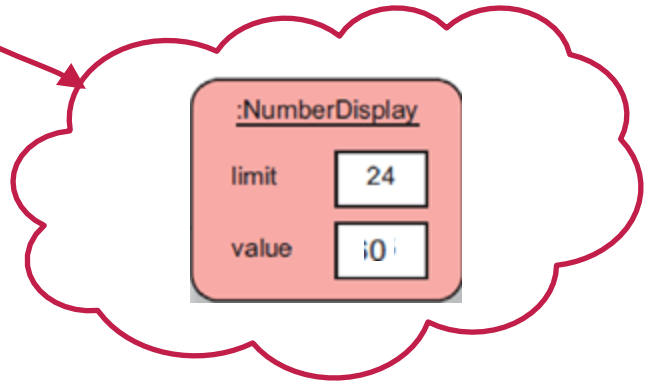
Tipos de datos Objeto

Dirección	Identificador	Valor
#0		`null`
#1	`hours`	
#2		
#3		
#4		
#5		

Inicialización

```
NumberDisplay hours = new NumberDisplay(24);
```

- La referencia apunta al nuevo objeto creado



Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

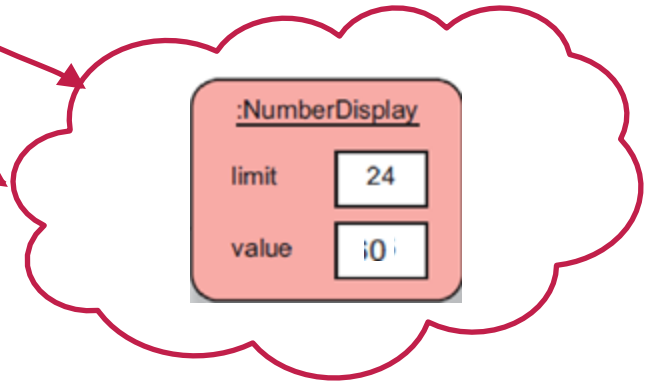
Gestión Interna de la Memoria

Tipos de datos Objetos

Dirección	Identificador	Valor
#0		null
#1	hours	
#2	display2	
#3		
#4		
#5		

Asignación

```
NumberDisplay hours = new NumberDisplay(24);
NumberDisplay display2 = hours;
```



Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

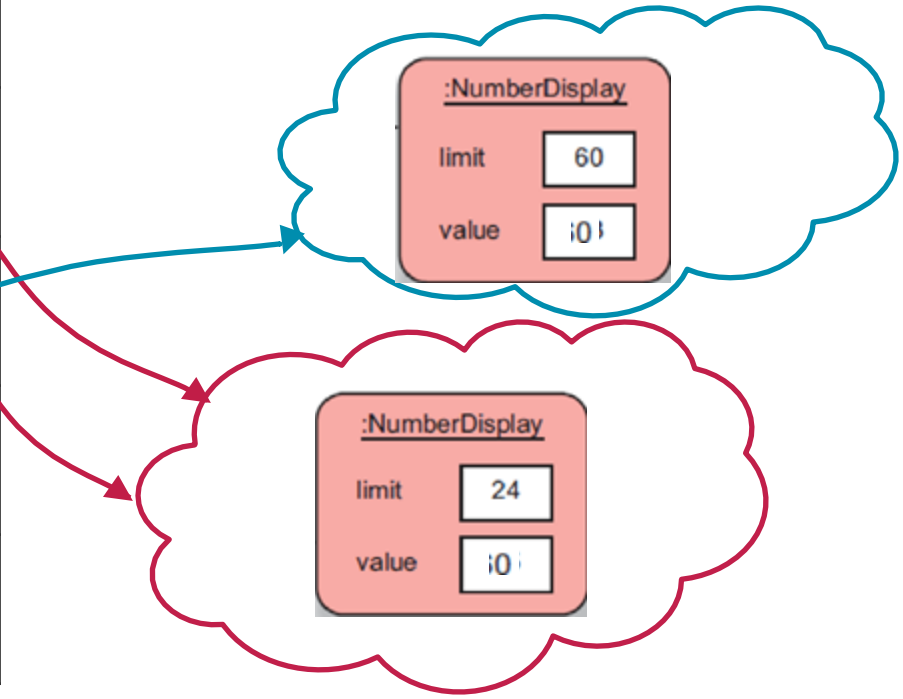
Gestión Interna de la Memoria

Tipos de datos Objeto

Dirección	Identificador	Valor
#0		null
#1	hours	
#2	display2	
#3	minutes	
#4		
#5		

Inicialización

```
NumberDisplay hours = new NumberDisplay(24);
NumberDisplay display2 = hours;
NumberDisplay minutes = new NumberDisplay(60);
```



Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

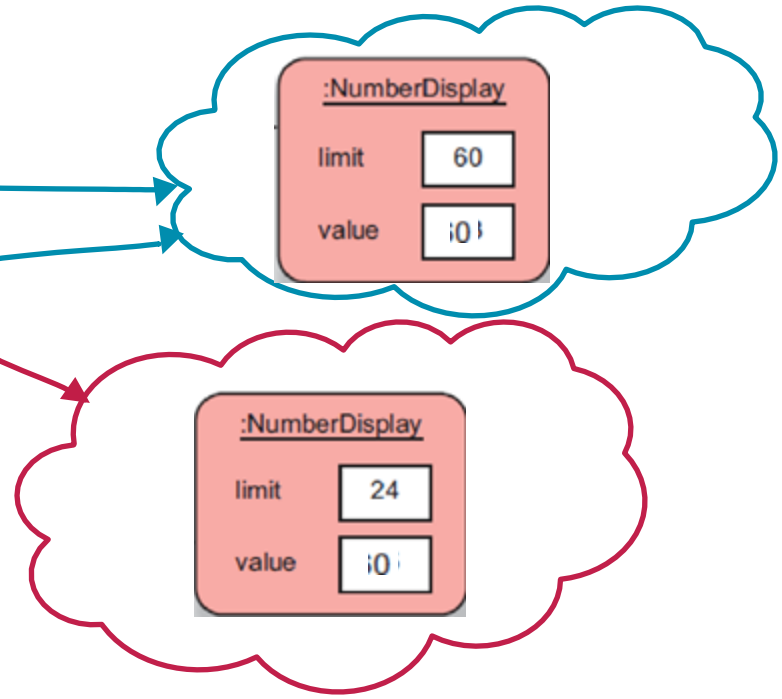
Gestión Interna de la Memoria

Tipos de datos Objeto

Dirección	Identificador	Valor
#0		null
#1	hours	
#2	display2	
#3	minutes	
#4		
#5		

Asignación

```
NumberDisplay hours = new NumberDisplay(24);
NumberDisplay display2 = hours;
NumberDisplay minutes = new NumberDisplay(60);
display2 = minutes;
```



Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Conceptos fundamentales

- modularización y abstracción
- tipos de datos: Primitivos y Objetos
- diagrama de clase
- creación de objetos
- diagrama de objetos
- sobrecarga de métodos
- interacción de objetos
- ocultando información
- Null
  - valores por defecto
- gestión interna de la memoria
  - tipos de datos primitivos
  - tipos de datos objeto
  - recolector de basura
  - tipos de datos primitivos – paso por valor
  - tipos de datos objeto – ¿paso por referencia?
- bibliotecas de Clases
- clase String
  - Identidad vs Igualdad
- documentación de clases
- uso de Javadoc para escribir documentación
- keyword this

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

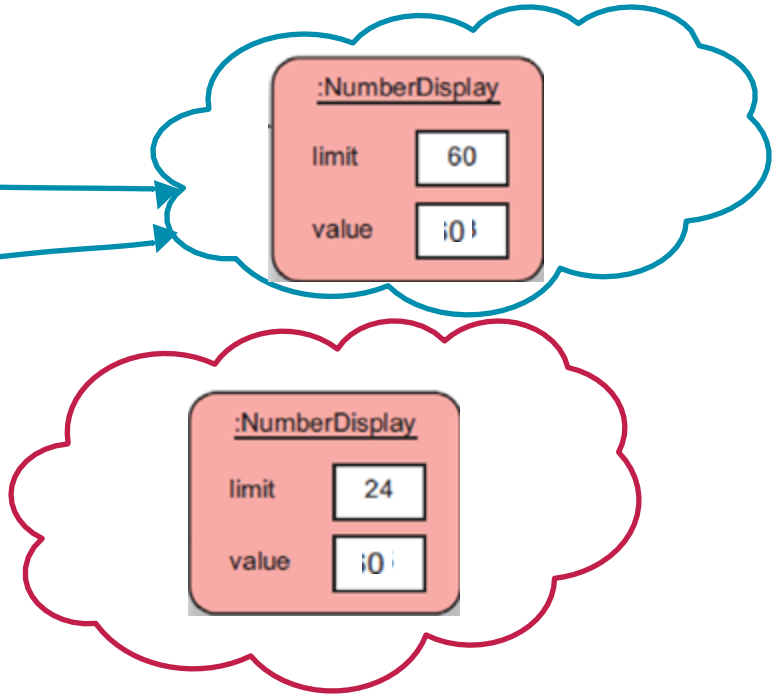
Gestión Interna de la Memoria

Recolector de basura

Dirección	Identificador	Valor
#0		null
#1	hours	
#2	display2	
#3	minutes	
#4		
#5		

Eliminación

```
NumberDisplay hours = new NumberDisplay(24);
NumberDisplay display2 = hours;
NumberDisplay minutes = new NumberDisplay(60);
display2 = minutes;
hours = null;
```



Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

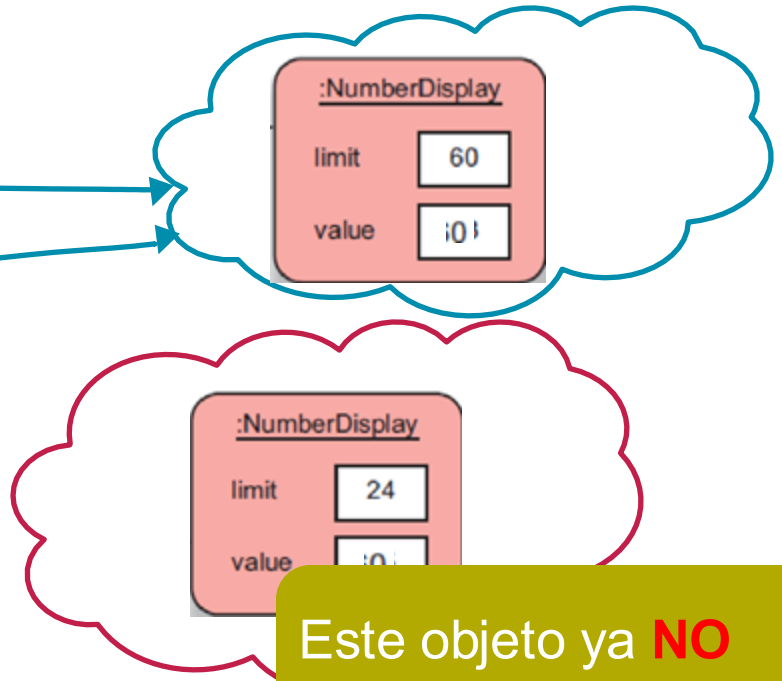
Gestión Interna de la Memoria

Recolector de basura

Dirección	Identificador	Valor
#0		null
#1	hours	
#2	display2	
#3	minutes	
#4		
#5		

Eliminación

```
NumberDisplay hours = new NumberDisplay(24);
NumberDisplay display2 = hours;
NumberDisplay minutes = new NumberDisplay(60);
display2 = minutes;
hours = null;
```



Este objeto ya **NO** puede ser accedido

Pearson
Copyright © 2017, 2012, 2009 Pearson Educat

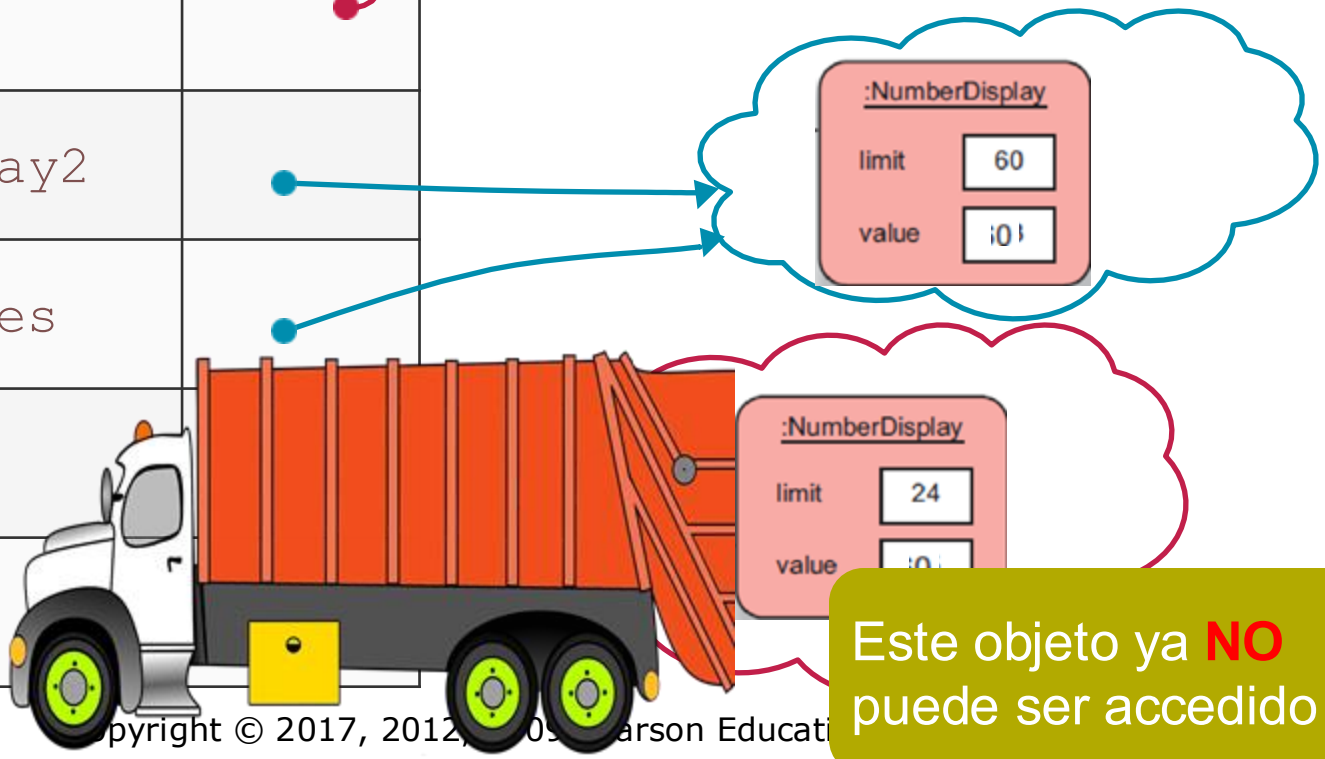
Gestión Interna de la Memoria

Recolector de basura

Dirección	Identificador	Valor
#0		null
#1	hours	
#2	display2	
#3	minutes	
#4		
#5		

Eliminación

```
NumberDisplay hours = new NumberDisplay(24);
NumberDisplay display2 = hours;
NumberDisplay minutes = new NumberDisplay(60);
display2 = minutes;
hours = null;
```



Este objeto ya **NO** puede ser accedido

Pearson
Copyright © 2017, 2012, 2009 Pearson Educati

Conceptos fundamentales

- modularización y abstracción
- tipos de datos: Primitivos y Objetos
- diagrama de clase
- creación de objetos
- diagrama de objetos
- sobrecarga de métodos
- interacción de objetos
- ocultando información
- Null
  - valores por defecto
- gestión interna de la memoria
  - tipos de datos primitivos
  - tipos de datos objeto
  - recolector de basura
  - tipos de datos primitivos – paso por valor
  - tipos de datos objeto – ¿paso por referencia?
- bibliotecas de Clases
- clase String
  - Identidad vs Igualdad
- documentación de clases
- uso de Javadoc para escribir documentación
- keyword this

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Gestión Interna de la Memoria

Tipos de datos Primitivos – Paso por valor

Dirección	Identificador	Valor
#0		
#1		
#2		
#3		
#4		
#5		

```
1   void doStuff() {
2      int num = 5;
3      addTwo(num);
4      System.out.println(num);
5   }
6   int addTwo(int number) {
7      number+=2;
8      return number;
9   }
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Gestión Interna de la Memoria

Tipos de datos Primitivos – Paso por valor

Dirección	Identificador	Valor
#0		
#1	num	5
#2		
#3		
#4		
#5		

```
1   void doStuff() {
2      int num = 5;
3      addTwo(num);
4      System.out.println(num);
5   }
6   int addTwo(int number) {
7      number+=2;
8      return number;
9   }
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Gestión Interna de la Memoria

Tipos de datos Primitivos – Paso por valor

Dirección	Identificador	Valor
#0		
#1	num	5
#2	number	5
#3		
#4		
#5		

```
1   void doStuff() {
2      int num = 5;
3      addTwo(num);
4      System.out.println(num);
5   }
6   int addTwo(int number) {
7      number+=2;
8      return number;
9   }
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Gestión Interna de la Memoria

Tipos de datos Primitivos – Paso por valor

Dirección	Identificador	Valor
#0		
#1	num	5
#2	number	7
#3		
#4		
#5		

```
1   void doStuff() {
2      int num = 5;
3      addTwo(num);
4      System.out.println(num);
5   }
6   int addTwo(int number) {
7      number+=2;
8      return number;
9   }
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Gestión Interna de la Memoria

Tipos de datos Primitivos – Paso por valor

Dirección	Identificador	Valor
#0		
#1	num	5
#2	number	7
#3		
#4		
#5		

```
1   void doStuff() {
2      int num = 5;
3      addTwo(num);
4      System.out.println(num);
5   }
6   int addTwo(int number) {
7      number+=2;
8      return number;
9   }
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Gestión Interna de la Memoria

Tipos de datos Primitivos – Paso por valor

Dirección	Identificador	Valor
#0		
#1	num	5
#2	number	7
#3		
#4		
#5		



```
1   void doStuff() {
2      int num = 5;
3      addTwo(num);
4      System.out.println(num);
5   }
6   int addTwo(int number) {
7     number+=2;
8     return number;
9   }
```



Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Gestión Interna de la Memoria

Tipos de datos Primitivos – Paso por valor

Dirección	Identificador	Valor
#0		
#1	num	5
#2		
#3		
#4		
#5		



```
1   void doStuff() {
2      int num = 5;
3      num = addTwo(num);
4      System.out.println(num);
5   }
6   int addTwo(int number) {
7      number+=2;
8      return number;
9   }
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Gestión Interna de la Memoria

Tipos de datos Primitivos – Paso por valor

Dirección	Identificador	Valor
#0		
#1	num	5
#2	number	5
#3		
#4		
#5		

```
1   void doStuff() {
2      int num = 5;
3      num = addTwo(num);
4      System.out.println(num);
5   }
6   int addTwo(int number) {
7      number+=2;
8      return number;
9   }
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Gestión Interna de la Memoria

Tipos de datos Primitivos – Paso por valor

Dirección	Identificador	Valor
#0		
#1	num	5
#2	number	7
#3		
#4		
#5		

```
1   void doStuff() {
2      int num = 5;
3      num = addTwo(num);
4      System.out.println(num);
5   }
6   int addTwo(int number) {
7      number+=2;
8      return number;
9   }
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Gestión Interna de la Memoria

Tipos de datos Primitivos – Paso por valor

Dirección	Identificador	Valor
#0		
#1	num	5
#2	number	7
#3		
#4		
#5		

```
1   void doStuff() {
2      int num = 5;
3      num = addTwo(num);
4      System.out.println(num);
5   }
6   int addTwo(int number) {
7      number+=2;
8      return number;
9   }
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Gestión Interna de la Memoria

Tipos de datos Primitivos – Paso por valor

Dirección	Identificador	Valor
#0		
#1	num	7
#2	number	7
#3		
#4		
#5		



```
1   void doStuff() {
2      int num = 5;
3      num = addTwo(num);
4      System.out.println(num);
5   }
6   int addTwo(int number) {
7      number+=2;
8      return number;
9   }
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Gestión Interna de la Memoria

Tipos de datos Primitivos – Paso por valor

Dirección	Identificador	Valor
#0		
#1	num	7
#2	number	7
#3		
#4		
#5		

```
1   void doStuff() {
2      int num = 5;
3      num = addTwo(num);
4      System.out.println(num);
5   }
6   int addTwo(int number) {
7     number+=2;
8     return number;
9   }
```





Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Conceptos fundamentales

- modularización y abstracción
- tipos de datos: Primitivos y Objetos
- diagrama de clase
- creación de objetos
- diagrama de objetos
- sobrecarga de métodos
- interacción de objetos
- ocultando información
- Null
  - valores por defecto
- gestión interna de la memoria
  - tipos de datos primitivos
  - tipos de datos objeto
  - recolector de basura
  - tipos de datos primitivos – paso por valor
  - tipos de datos objeto – ¿paso por referencia?
- bibliotecas de Clases
- clase String
  - Identidad vs Igualdad
- documentación de clases
- uso de Javadoc para escribir documentación
- keyword this

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Gestión Interna de la Memoria

Tipos de datos Objeto – Paso por referencia

Dirección	Identificador	Valor
#0		
#1		
#2		
#3		
#4		
#5		

```
1  class Celebration {
2   void static void main(String[] args){
3    Minion stuart = new Minion(5753);
4    Birthday bDay = new Birthday();
5    System.out.println(stuart.getAge());
6    bDay.celebrate(stuart);
7    System.out.println(stuart.getAge());
8   }
9  }
10 class Birthday {
11  void celebrate(Minion m) {
12    m.setAge(m.getAge()+1);
13  }
14 }
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Gestión Interna de la Memoria

Tipos de datos Objeto – Paso por referencia

Dirección	Identificador	Valor
#0		
#1	stuart	
#2		
#3		
#4		
#5		




```
1  class Celebration {
2   void static void main(String[] args){
3    Minion stuart = new Minion(5753);
4    Birthday bDay = new Birthday();
5    System.out.println(stuart.getAge());
6    bDay.celebrate(stuart);
7    System.out.println(stuart.getAge());
8   }
9  }
10 class Birthday {
11  void celebrate(Minion m) {
12    m.setAge(m.getAge()+1);
13  }
14 }
```

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Gestión Interna de la Memoria

Tipos de datos Objeto – Paso por referencia

Dirección	Identificador	Valor
#0		
#1	stuart	
#2	bDay	
#		
#4		
#5		






```
1  class Celebration {
2   void static void main(String[] args){
3    Minion stuart = new Minion(5753);
4    Birthday bDay = new Birthday();
5    System.out.println(stuart.getAge());
6    bDay.celebrate(stuart);
7    System.out.println(stuart.getAge());
8   }
9  }
```

```
10 class Birthday {
11  void celebrate(Minion m) {
12    m.setAge(m.getAge()+1);
13  }
14 }
```

ight © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Gestión Interna de la Memoria

Tipos de datos Objeto – Paso por referencia

Dirección	Identificador	Valor
#0		
#1	stuart	
#2	bDay	
#3		
#4		
#5		






```
1  class Celebration {
2   void static void main(String[] args){
3    Minion stuart = new Minion(5753);
4    Birthday bDay = new Birthday();
5    System.out.println(stuart.getAge());
6    bDay.celebrate(stuart);
7    System.out.println(stuart.getAge());
8   }
9  }
```

```
10 class Birthday {
11  void celebrate(Minion m) {
12    m.setAge(m.getAge()+1);
13  }
14 }
```

```
>_ 5753
```

Copyright © 2017, ...

Gestión Interna de la Memoria

Tipos de datos Objeto – Paso por referencia

Dirección	Identificador	Valor
#0		
#1	stuart	
#2	bDay	
#3		
#4		
#5		






```
1  class Celebration {
2   void static void main(String[] args){
3    Minion stuart = new Minion(5753);
4    Birthday bDay = new Birthday();
5    System.out.println(stuart.getAge());
6    bDay.celebrate(stuart);
7    System.out.println(stuart.getAge());
8   }
9  }
10 class Birthday {
11   void celebrate(Minion m) {
12     m.setAge(m.getAge()+1);
13   }
14 }
```

```
>_ 5753
```

ight © 2017, 7

Gestión Interna de la Memoria

Tipos de datos Objeto – Paso por referencia

Dirección	Identificador	Valor
#0		
#1	stuart	
#2	bDay	
#3		
#4		
#5		




```
1  class Celebration {
2   void static void main(String[] args){
3    Minion stuart = new Minion(5753);
4    Birthday bDay = new Birthday();
5    System.out.println(stuart.getAge());
6    bDay.celebrate(stuart);
7    System.out.println(stuart.getAge());
8   }
9  }
```

```
10 class Birthday {
11   void celebrate(Minion m) {
12     m.setAge(m.getAge()+1);
13   }
14 }
```

```
>_ 5753
```

ight © 2017, 7

Gestión Interna de la Memoria

Tipos de datos Objeto – Paso por referencia

Dirección	Identificador	Valor
#0		
#1	stuart	
#2	bDay	
#3		
#4		
#5		




```
1  class Celebration {
2   void static void main(String[] args){
3    Minion stuart = new Minion(5753);
4    Birthday bDay = new Birthday();
5    System.out.println(stuart.getAge());
6    bDay.celebrate(stuart);
7    System.out.println(stuart.getAge());
8   }
9  }
```

```
10 class Birthday {
11   void celebrate(Minion m) {
12     m.setAge(m.getAge()+1);
13   }
14 }
```

```
>_ 5753
>_ 5754
```

ight © 2017,

Conceptos fundamentales

- modularización y abstracción
- tipos de datos: Primitivos y Objetos
- diagrama de clase
- creación de objetos
- diagrama de objetos
- sobrecarga de métodos
- interacción de objetos
- ocultando información
- Null
  - valores por defecto
- gestión interna de la memoria
  - tipos de datos primitivos
  - tipos de datos objeto
  - recolector de basura
  - tipos de datos primitivos – paso por valor
  - tipos de datos objeto – ¿paso por referencia?
- bibliotecas de Clases
- clase String
  - Identidad vs Igualdad
- documentación de clases
- uso de Javadoc para escribir documentación
- keyword this

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Bibliotecas de Clases

- Conjunto de **clases útiles**.
  - p.ej. String
- No tenemos que escribir todo desde cero.
- Las bibliotecas de clases contienen clases probadas y comprobadas.
- Java llama paquetes (**packages**) a sus bibliotecas.
- Usalas del mismo modo que las clases que implementes (es decir, sus constructores y métodos)
  - ¡¡ Ojo, independientemente del número de clases de la Biblioteca Java que uses, **nunca aparecerán** en el diagrama de clases de BlueJ!!

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Bibliotecas de Clases

- https://docs.oracle.com/en/java/javase/21/
- https://docs.oracle.com/en/java/javase/21/docs/api/index.html
- https://docs.oracle.com/javase/tutorial/collections/

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Bibliotecas de Clases

Trabajando con la Biblioteca

- Una persona que programe bien debería trabajar con bibliotecas de clases:
  - conociendo el nombre de algunas clases importantes;
  - sabiendo encontrar otras clases.
- Recuerda:
  - sólo necesitamos conocer la **interfaz***,

    no la **implementación (*implementation*)****.

(*) Documentación, constructor público y cabeceras de los métodos

(**) Campos, constructores y métodos privados y cuerpo de código

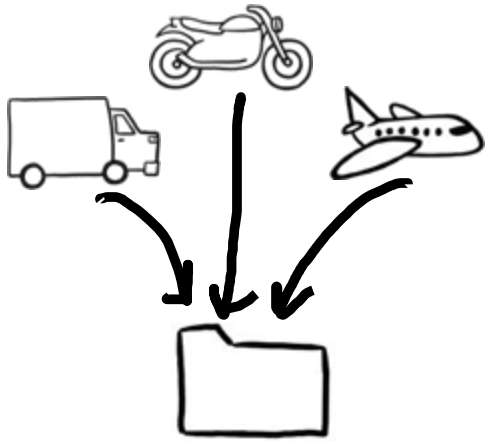
Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Bibliotecas de Clases

Paquetes (packages) e Import

- Las clases se organizan en paquetes (*packages*).
  - Espacio de nombres (*namespace*) que organiza un conjunto de clases
  - Similar a carpetas o directorios en un ordenador
- Las clases de la librería deben ser **importadas** con la sentencia ***`import`***;
  - Excepto para el paquete **`java.lang`**.
  - Pueden importarse clases simples:
    ```
    import java.util.ArrayList;
    ```
  - Pueden importarse paquetes completos:
    ```
    import java.util.*;
    ```



- Una vez importadas pueden usarse como clases en el proyecto.
  - Importar no implica la inserción del código

Conceptos fundamentales

- modularización y abstracción
- tipos de datos: Primitivos y Objetos
- diagrama de clase
- creación de objetos
- diagrama de objetos
- sobrecarga de métodos
- interacción de objetos
- ocultando información
- Null
  - valores por defecto
- gestión interna de la memoria
  - tipos de datos primitivos
  - tipos de datos objeto
  - recolector de basura
  - tipos de datos primitivos – paso por valor
  - tipos de datos objeto – ¿paso por referencia?
- bibliotecas de Clases
- clase String
  - Identidad vs Igualdad
- documentación de clases
- uso de Javadoc para escribir documentación
- keyword this

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

La Clase String

- Está definida en el paquete **`java.lang`**.
- Tiene algunas características que la diferencian de otras clases con las que hay que tener cuidado:
  - Son inmutables
  - La comparación de objetos **`String`** puede darnos sorpresas inesperadas.
    - Para no llevarnos esas sorpresas necesitamos saber la diferencia entre **Identidad e Igualdad**

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

La Clase String

Inmutabilidad de String

Método de ejemplo de *String*

```
String toUpperCase()
```

Uso incorrecto:

```
input.toUpperCase();
```

Usos correctos:

```
input = input.toUpperCase();
```

```
if(input.toUpperCase().contains())…
```

¡Cuidado! Cuando se invoca a sus métodos, se crea un nuevo objeto String con el resultado de la modificación.

Este resultado lo podemos asignar a la variable que almacenaba la referencia al String original o a una nueva)

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Conceptos fundamentales

- modularización y abstracción
- tipos de datos: Primitivos y Objetos
- diagrama de clase
- creación de objetos
- diagrama de objetos
- sobrecarga de métodos
- interacción de objetos
- ocultando información
- Null
  - valores por defecto
- gestión interna de la memoria
  - tipos de datos primitivos
  - tipos de datos objeto
  - recolector de basura
  - tipos de datos primitivos – paso por valor
  - tipos de datos objeto – paso por referencia
- bibliotecas de Clases
- clase String
  - Identidad vs Igualdad
- documentación de clases
- uso de Javadoc para escribir documentación
- keyword this

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Identidad Vs Igualdad



Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Identidad Vs Igualdad 1

Con objetos que **NO** son de tipo **String**:

Dirección	Identificador	Valor
#0		null
#1	stuart1	
#2	stuart2	
#3		
#4		
#5		

```
1 Minion stuart1=new Minion(5753);
2 Minion stuart2=new Minion(5753);
3
4 System.out.println(stuart1==stuart2);
```




```
false
```

Pearson

Copyright © 2017, . All Rights Reserved

Identidad Vs Igualdad 2

Con objetos que **NO** son de tipo **String**:

Dirección	Identificador	Valor
#0		null
#1	stuart1	
#2	stuart2	
#3		
#4		
#5		

```
1 Minion stuart1=new Minion(5753);
2 Minion stuart2=new Minion(5754);
3 stuart2=stuart1;
4 System.out.println(stuart1==stuart2);
```






```
true
```

Pearson

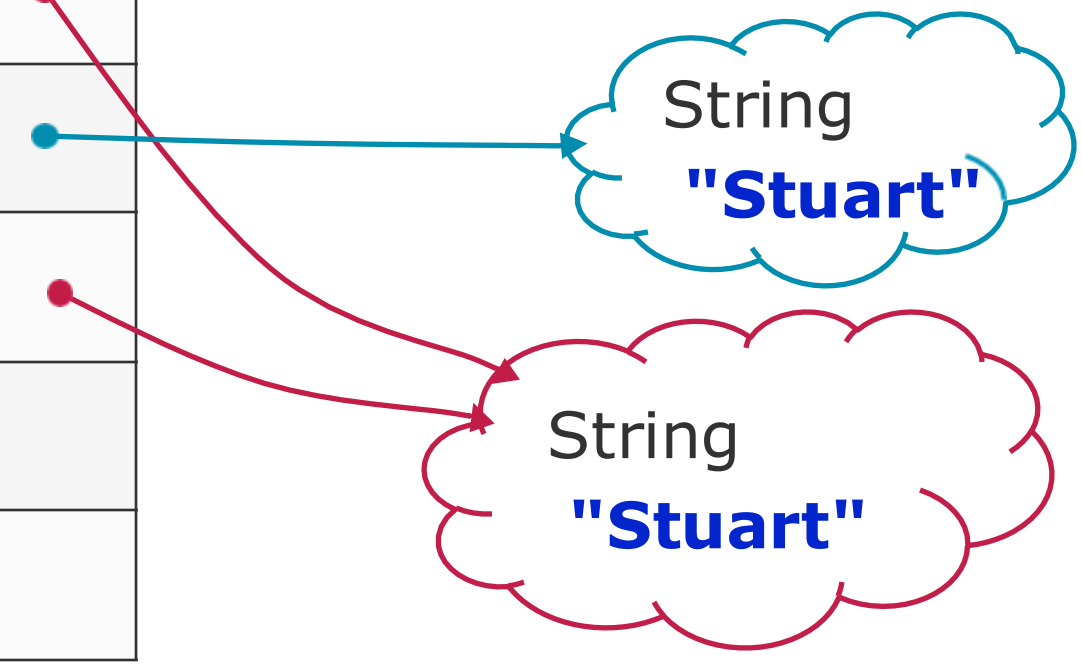
Copyright © 2017, . All Rights Reserved

Identidad Vs Igualdad 3

Con objetos tipo **String**:

Addr.	Identifier	Value
#0		null
#1	st1	
#2	st2	
#3	st3	
#4		
#5		

```
String st1 = new String("Stuart");
String st2 = new String("Stuart");
String st3 = st1;
```



Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

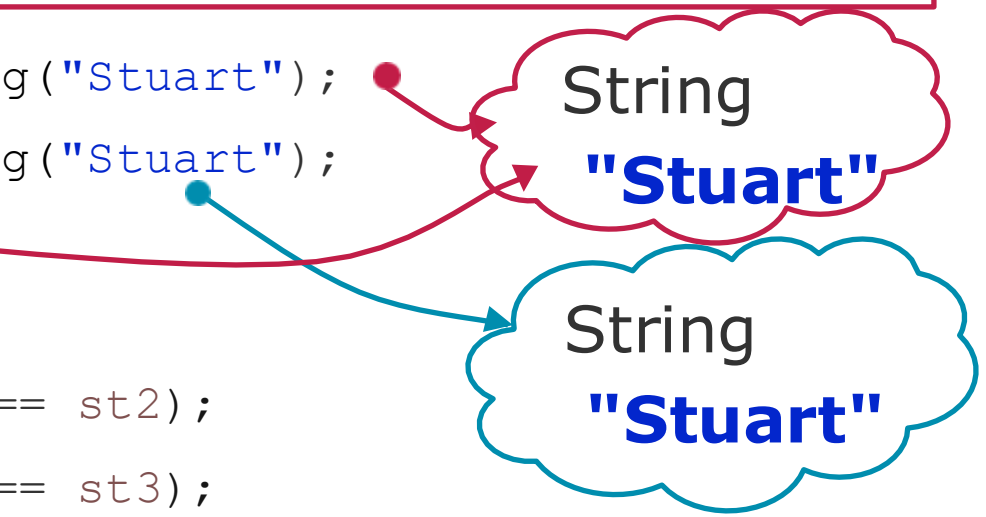
Identidad Vs Igualdad 4

Con objetos tipo **String**:

Comparación con ==

comprueba si las **referencias** apuntan al **mismo objeto**

```
1    String st1 = new String("Stuart");
2    String st2 = new String("Stuart");
3    String st3 = st1;
4
5    System.out.println(st1 == st2);
6    System.out.println(st1 == st3);
```




```
>_
false
true
```

Hasta ahora todo parece igual que con objetos no tipo String, ¿o no…?

Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Problema con la Comparación de Strings

- Los `String` que tienen un valor asignado de forma literal (no se ha creado el objeto de forma explícita con new) se comportan de forma distinta cuando se comparan usando ==
- Esto es debido a que el compilador mezcla los literales idénticos de `String` que encuentra en el código del programa.
  - El resultado es que en una comparación podemos obtener un resultado afirmativo comprobando igualdad con aparentemente distintos objetos `String`.
- Pero en cambio no lo hace para objetos idénticos que aparecen fuera del código del programa;
  - p.ej., si los ha introducido el usuario por teclado.

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Identidad Vs Igualdad 5

Con objetos tipo **String literales**:

```
1   String     stuart1        =       "Stuart";
2   String     stuart2        =       "Stuart";
3
4   System.out.println(stuart1 == stuart2);
```

```
> true
```

stuart1 == stuart2

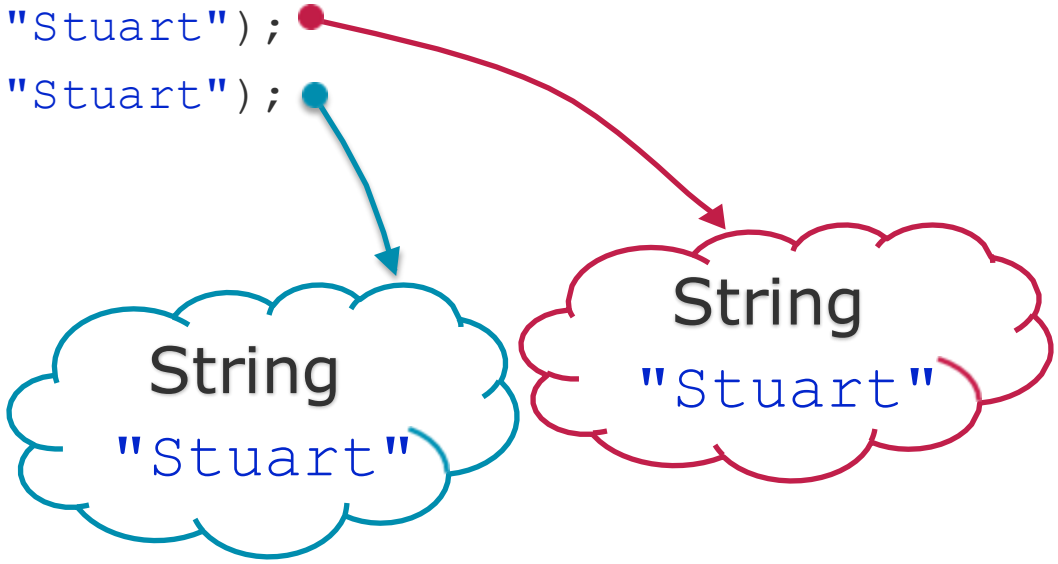
Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

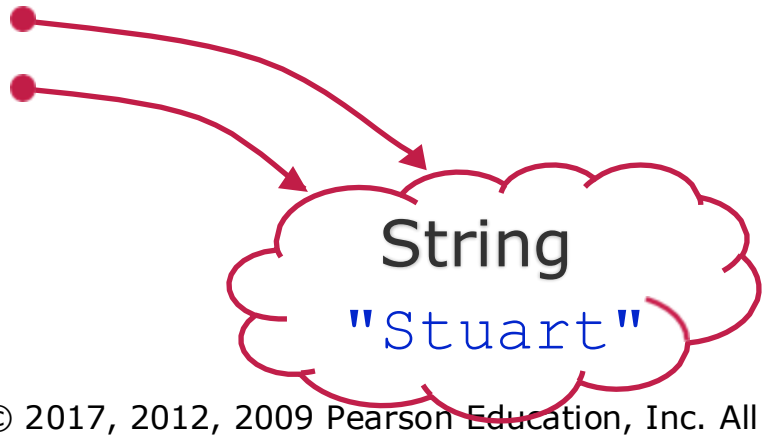
Identidad Vs Igualdad 6

Con objetos tipo **String** y **String literales**:

```
String st1 = new String("Stuart");
String st2 = new String("Stuart");
```




```
String stuart1 = "Stuart";
String stuart2 = "Stuart";
```



Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

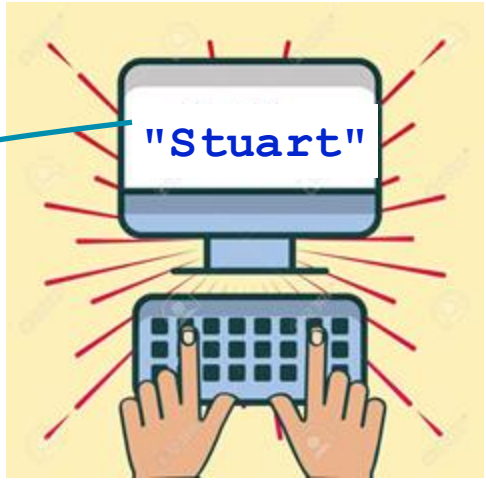
Problema con la Comparación de Strings

- Pero en cambio no mezcla los literales idénticos de `String` que aparecen fuera del código del programa;
  - p.ej., si los ha introducido el usuario por teclado.

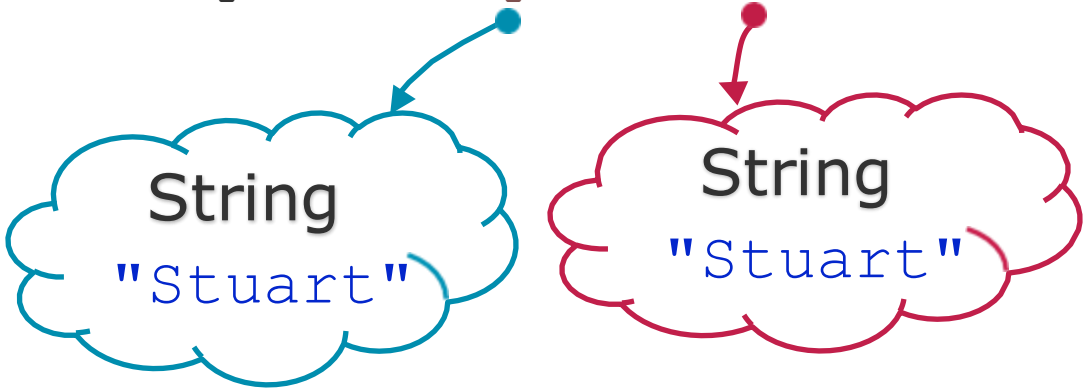
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Identidad Vs Igualdad 6

Con objetos tipo **String literales**:




```
1   String input = reader.getInput();
2   System.out.println(input == "Stuart");
```




```
> false
```

input == "Stuart"

Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Solución

- Necesitamos comparar la igualdad del contenido para evitar resultados no deseados
  - no tan solo si sus referencias son idénticas

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

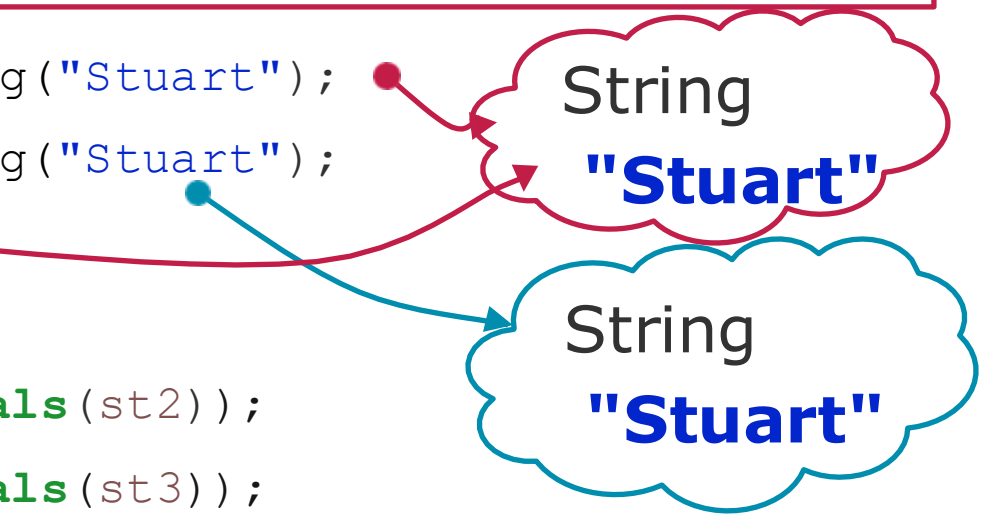
Identidad Vs Igualdad 7

Con objetos tipo **String**:

Comparación con `equals()`

comprueba si los **objetos** contienen los **mismos valores**

```
1      String  st1  =  New String("Stuart");
2      String  st2  =  new String("Stuart");
3      String  st3  =  st1;
4
5   System.out.println(st1.equals(st2));
6   System.out.println(st1.equals(st3));
```




```
>_
true
true
```

Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Identidad Vs Igualdad 8

Con objetos tipo **String literales**:

Comparación con `equals()`

comprueba si los **objetos** contienen los **mismos valores**

```
1   String stuart1 = "Stuart";
2   String stuart2 = "Stuart";
3
4   System.out.println(stuart1.equals(stuart2));
```

```
> __  true   <-------- stuart1.equals(stuart2)
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

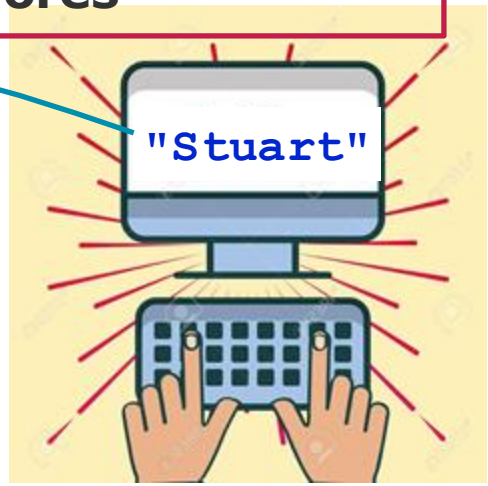
Identidad Vs Igualdad 9

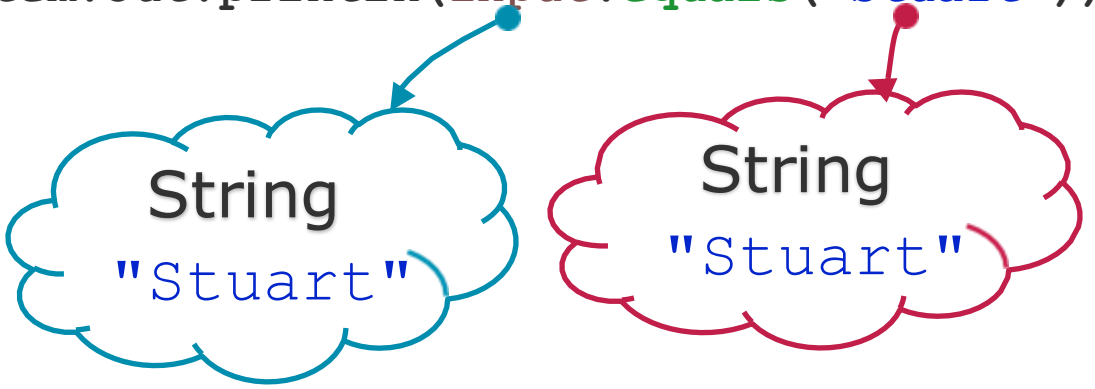
Con objetos tipo **String literales**:

Comparación con `equals()`

comprueba si los **objetos** contienen los **mismos valores**

```
1   String input = reader.getInput();
2   System.out.println(input.equals("Stuart"));
```






```
> true  <-- input.equals("Stuart")
```

Pearson
Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Conclusión sobre Strings

¿Son el *mismo* String?

```
input == "Stuart"
```

Chequeo de identidad

¡¡No usar!!

```
input.equals("Stuart")
```

Chequeo de igualdad

¿Tienen los *mismos* caracteres?

Importante:
¡¡usa siempre **equals*** para comprobar la igualdad de String!!

*Para poder usar **equals** con nuestros propios objetos no String tendremos que hacer algo más que veremos en una unidad posterior

Conceptos fundamentales

- modularización y abstracción
- tipos de datos: Primitivos y Objetos
- diagrama de clase
- creación de objetos
- diagrama de objetos
- sobrecarga de métodos
- interacción de objetos
- ocultando información
- Null
  - valores por defecto
- gestión interna de la memoria
  - tipos de datos primitivos
  - tipos de datos objeto
  - recolector de basura
  - tipos de datos primitivos – paso por valor
  - tipos de datos objeto – ¿paso por referencia?
- bibliotecas de Clases
- clase String
  - Identidad vs Igualdad
- documentación de clases
- uso de Javadoc para escribir documentación
- keyword this

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Documentación de clases

- La documentación de la biblioteca de clases de Java muestra los detalles de todas las clases de la biblioteca.
  - Se lee en un navegador web (Está en formato HTML)
- Es la descripción de la API (***Application Programmers Interface***) de todas las clases de la biblioteca.
- Saber usar esta documentación es esencial para poder hacer un buen uso de las clases

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Interfaz vs *Implementación* (1 de 2)

La documentación incluye (QUÉ)

- el nombre de la clase;
- una descripción general de la clase;
- una lista de constructores y métodos
- los valores de retorno (*return*) y parámetros para los constructores y métodos
- una descripción del propósito de cada constructor y método



interfaz de la clase

La ***interfaz*** de una clase describe lo que la clase hace y cómo puede ser usada sin mostrar la implementación

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Interfaz vs *Implementación* (2 de 2)

La documentación **no** incluye (**CÓMO**)

- campos private (la mayoría lo son)
- métodos private
- el cuerpo (código fuente) de los métodos



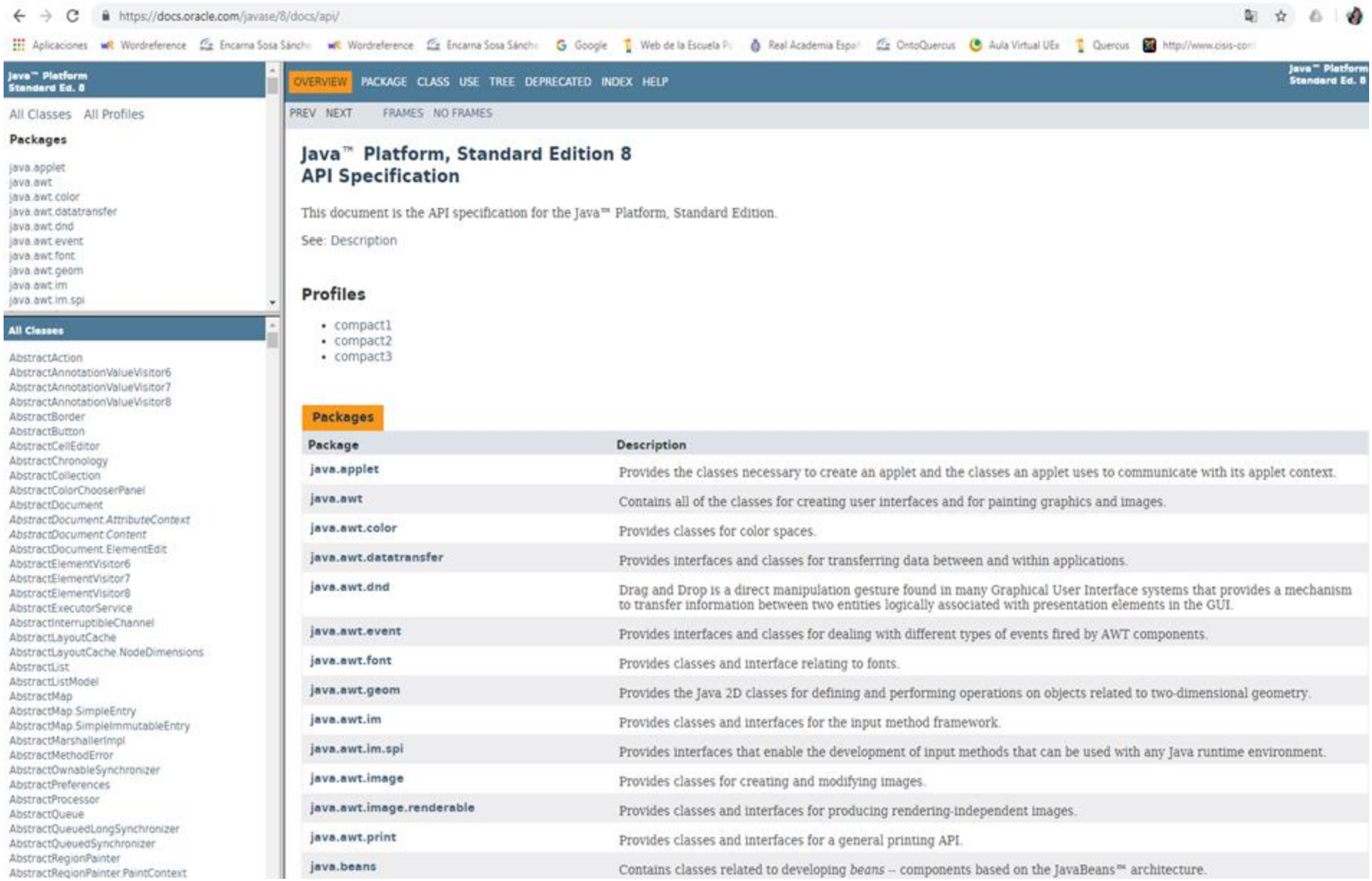
la **implementación** de la clase

La ***implementación*** de una clase es el Código Fuente al completo de una clase

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

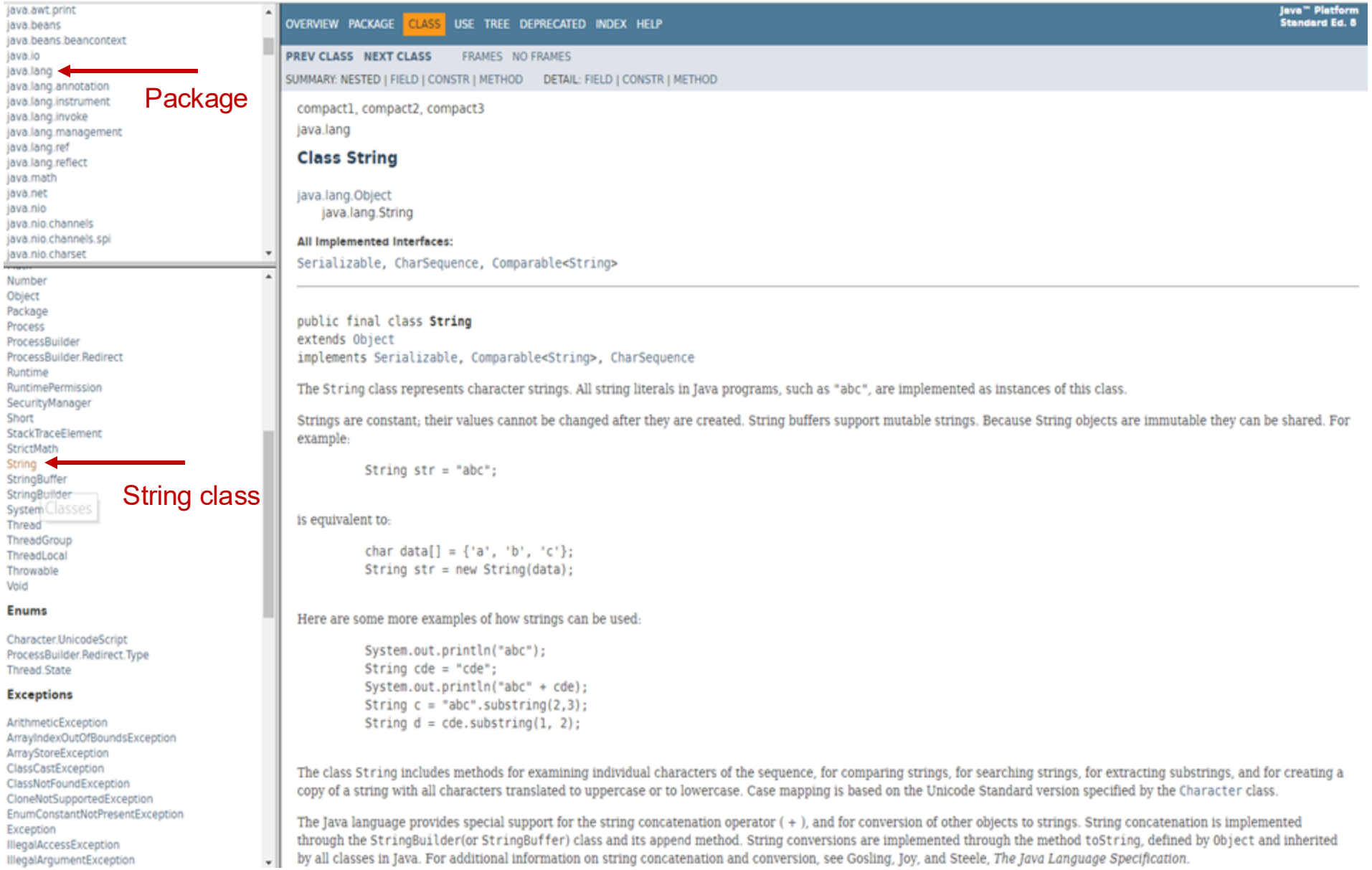
Referencia API (https://docs.oracle.com/en/java/javase/17/docs/api/index.html)



Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

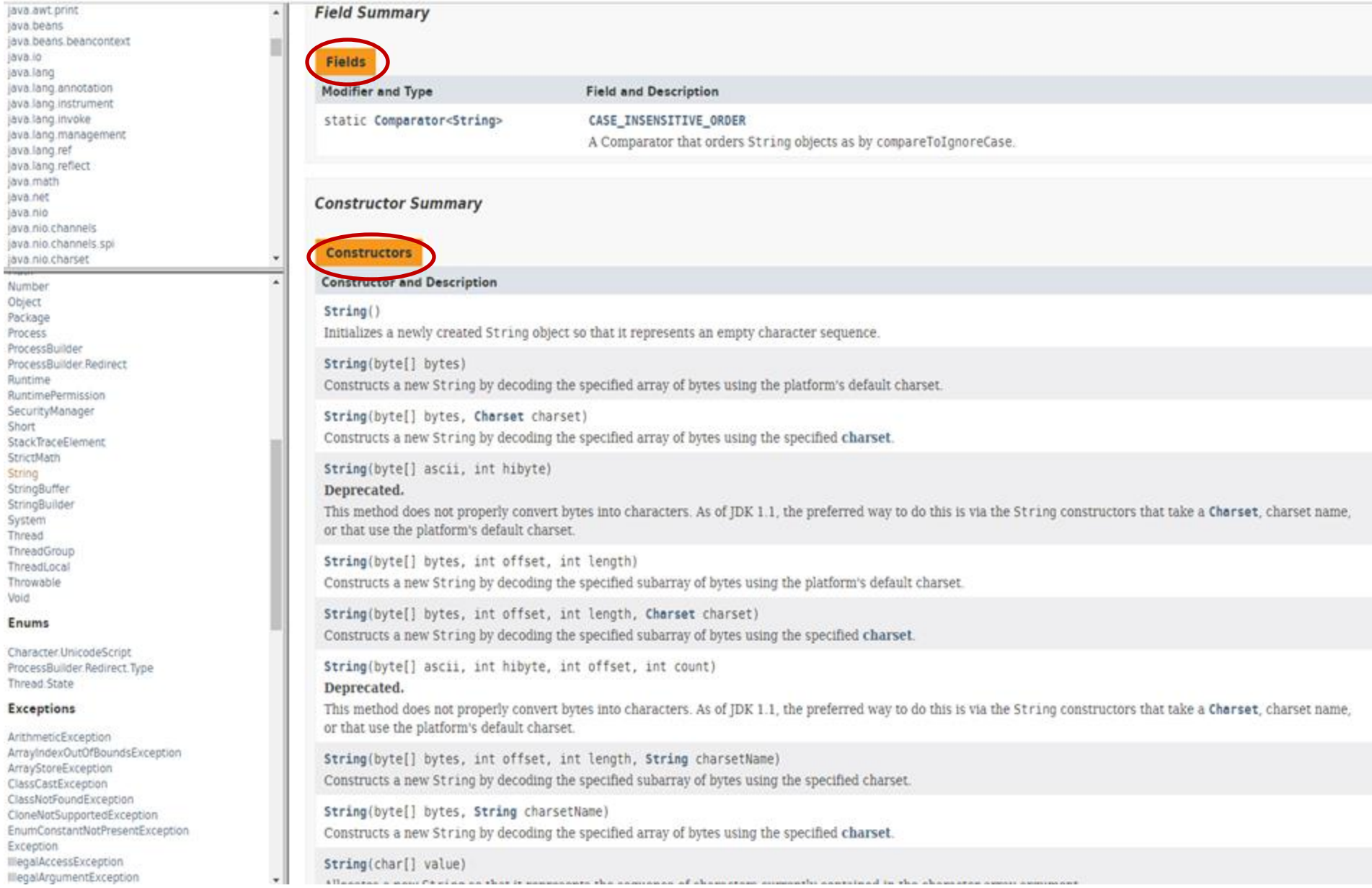
Referencia API (clase *String*)



Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

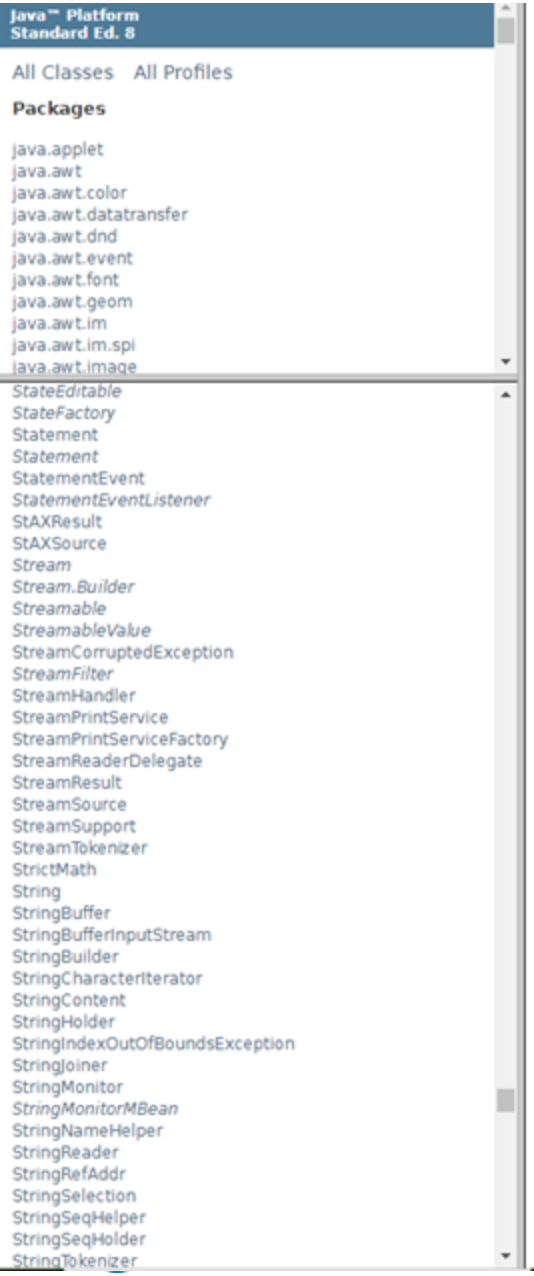
Referencia API (clase *String*)



Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Referencia API (clase *String*)



Method Summary

All Methods | Static Methods | Instance Methods | Concrete Methods | Deprecated Methods

Modifier and Type	Method and Description
char	**charAt**(int index) Returns the char value at the specified index.
int	**codePointAt**(int index) Returns the character (Unicode code point) at the specified index.
int	**codePointBefore**(int index) Returns the character (Unicode code point) before the specified index.
int	**codePointCount**(int beginIndex, int endIndex) Returns the number of Unicode code points in the specified text range of this String.
int	**compareTo**(String anotherString) Compares two strings lexicographically.
int	**compareToIgnoreCase**(String str) Compares two strings lexicographically, ignoring case differences.
String	**concat**(String str) Concatenates the specified string to the end of this string.
boolean	**contains**(CharSequence s) Returns true if and only if this string contains the specified sequence of char values.
boolean	**contentEquals**(CharSequence cs) Compares this string to the specified CharSequence.
boolean	**contentEquals**(StringBuffer sb) Compares this string to the specified StringBuffer.
static String	**copyValueOf**(char[] data) Equivalent to **valueOf**(char[]).
static String	**copyValueOf**(char[] data, int offset, int count) Equivalent to **valueOf**(char[], int, int).
boolean	**endsWith**(String suffix) Tests if this string ends with the specified suffix.
boolean	**equals**(Object anObject) Compares this string to the specified object.
boolean	**equalsIgnoreCase**(String anotherString) Compares this String to another String, ignoring case considerations.

Referencia API (clase *String*)

equals

```
public boolean equals(Object anObject)
```

Compares this string to the specified object. The result is `true` if and only if the argument is not `null` and is a `String` object that represents the same sequence of characters as this object.

Overrides:

`equals` in class `Object`

Parameters:

`anObject` - The object to compare this String against

Returns:

`true` if the given object represents a String equivalent to this string, false otherwise

See Also:

`compareTo(String)`, `equalsIgnoreCase(String)`

contains

```
public boolean contains(CharSequence s)
```

Returns true if and only if this string contains the specified sequence of char values.

Parameters:

`s` - the sequence to search for

Returns:

`true` if this string contains `s`, false otherwise

Since:

1.5

trim

```
public String trim()
```

Returns a string whose value is this string, with any leading and trailing whitespace removed.

If this `String` object represents an empty character sequence, or the first and last characters of character sequence represented by this `String` object both have codes greater than `'\u0020'` (the space character), then a reference to this `String` object is returned.

Otherwise, if there is no character with a code greater than `'\u0020'` in the string, then a `String` object representing an empty string is returned.

Otherwise, let *k* be the index of the first character in the string whose code is greater than `'\u0020'`, and let *m* be the index of the last character in the string whose code is greater than `'\u0020'`. A `String` object is returned, representing the substring of this string that begins with the character at index *k* and ends with the character at index *m*-that is, the result of `this.substring(k, m + 1)`.

This method may be used to trim whitespace (as defined above) from the beginning and end of a string.

Returns:

A string whose value is this string, with any leading and trailing white space removed, or this string if it has no leading or trailing white space.

Pearson Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Referencia API (clase *String*)

toUpperCase

```
public String toUpperCase()
```

Converts all of the characters in this String to upper case using the rules of the default locale. This method is equivalent to toUpperCase(Locale.getDefault()).

Note: This method is locale sensitive, and may produce unexpected results if used for strings that are intended to be interpreted locale independently. Examples are programming language identifiers, protocol keys, and HTML tags. For instance, `"title".toUpperCase()` in a Turkish locale returns `"T\u0130TLE"`, where '\u0130' is the LATIN CAPITAL LETTER I WITH DOT ABOVE character. To obtain correct results for locale insensitive strings, use `toUpperCase(Locale.ROOT)`.

Returns:

the String, converted to uppercase.

See Also:

toUpperCase(Locale)

startsWith

```
public boolean startsWith(String prefix)
```

Tests if this string starts with the specified prefix.

Parameters:

prefix - the prefix.

Returns:

true if the character sequence represented by the argument is a prefix of the character sequence represented by this string; false otherwise. Note also that true will be returned if the argument is an empty string or is equal to this String object as determined by the equals(Object) method.

Since:

1. 0

endsWith

```
public boolean endsWith(String suffix)
```

Tests if this string ends with the specified suffix.

Parameters:

suffix - the suffix.

Returns:

true if the character sequence represented by the argument is a suffix of the character sequence represented by this object; false otherwise. Note that the result will be true if the argument is the empty string or is equal to this String object as determined by the equals(Object) method.

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Conceptos fundamentales

- modularización y abstracción
- tipos de datos: Primitivos y Objetos
- diagrama de clase
- creación de objetos
- diagrama de objetos
- sobrecarga de métodos
- interacción de objetos
- ocultando información
- Null
  - valores por defecto
- gestión interna de la memoria
  - tipos de datos primitivos
  - tipos de datos objeto
  - recolector de basura
  - tipos de datos primitivos – paso por valor
  - tipos de datos objeto – ¿paso por referencia?
- bibliotecas de Clases
- clase String
  - Identidad vs Igualdad
- documentación de clases
- uso de Javadoc para escribir documentación
- keyword this

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Escribiendo documentación de clases

- Nuestras propias clases deben documentarse de la misma forma que la biblioteca de clases.
- ¡Haz de tus clases una potencial 'biblioteca de clases'!

> La ***documentación*** de una clase debería ser lo suficientemente detallada para que otros programadores usen la clase sin necesidad de leer su implementación

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Elementos en la documentación (1 de 2)

La documentación de una clase debería incluir:

- el nombre de la clase
- un comentario que describa el propósito general y las características de la clase
- un número de versión
- los nombres de los autores
- documentación para cada constructor y cada método

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Elementos en la documentación (2 de 2)

La documentación para cada constructor y método debería incluir:

- el nombre del método
- el tipo de retorno
- los nombres de parámetros y sus tipos
- una descripción del propósito y la función del método
- una descripción de cada parámetro
- una descripción del valor devuelto

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Javadoc

Escribiendo comentario de una clase:

```
/**
* The responder class represents a response generator object.
* It is used to generate an automatic response to an input
* string.
*
* @author      Michael Kölling and David J. Barnes
* @version     0.1 (2016.02.29)
*
*/

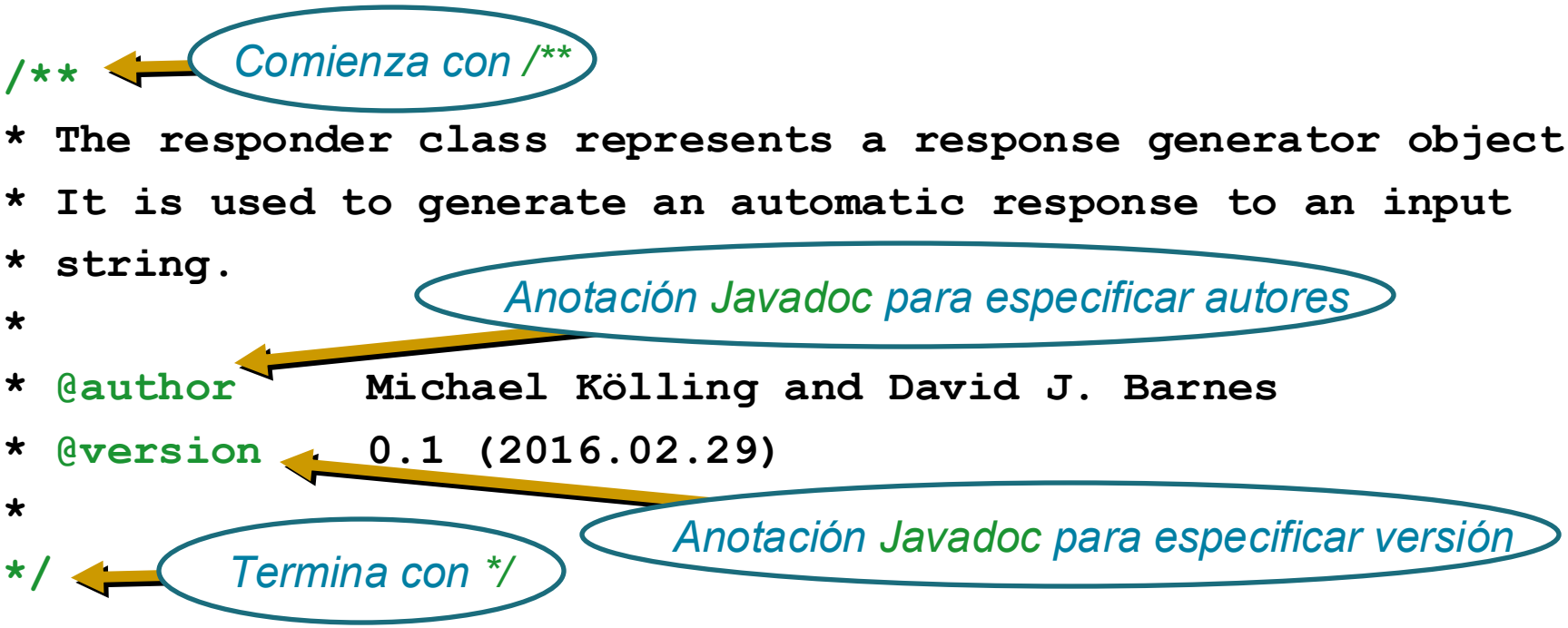
public class Responder  {
…
}
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Javadoc

Escribiendo comentario de una clase:




```
/**
* The responder class represents a response generator object.
* It is used to generate an automatic response to an input
* string.
*
* @author      Michael Kölling and David J. Barnes
* @version     0.1 (2016.02.29)
*
*/
public class Responder  {
…
}
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Javadoc

Escribiendo comentario de un método:

```
/**
* Read a line of text from standard input (the text terminal),
* and return it as a set of words.
*
* @param   prompt A prompt to print to screen
* @return  A set of Strings, where each String is one of the
*          words typed by the user
*
*/

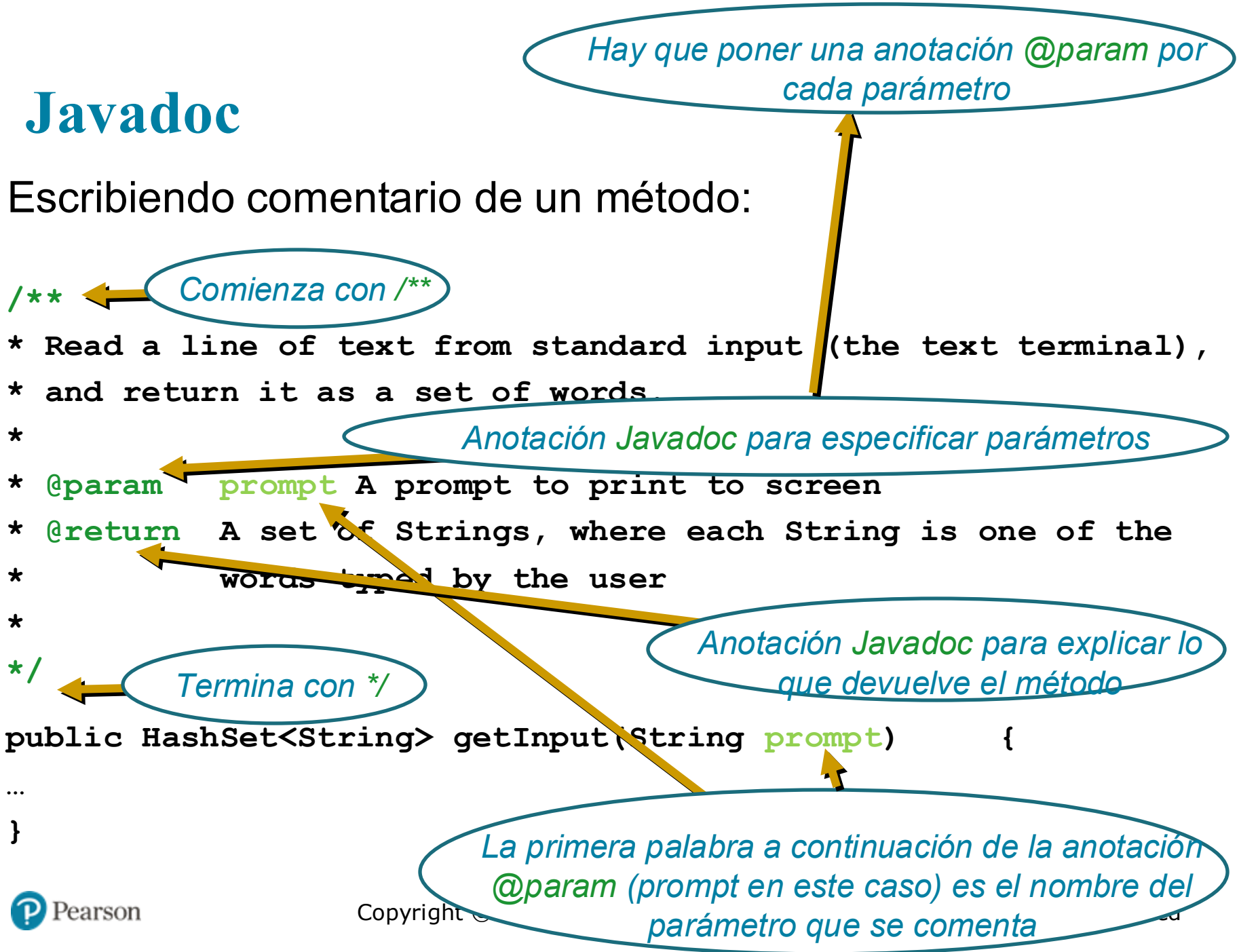
public HashSet<String> getInput(String prompt)      {
…
}
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Javadoc

Escribiendo comentario de un método:




```
/**
* Read a line of text from standard input (the text terminal),
* and return it as a set of words.
*
* @param    prompt A prompt to print to screen
* @return   A set of Strings, where each String is one of the
*           words typed by the user
*
*/
public HashSet<String> getInput(String prompt)      {
...
}
```


Copyright ©

Conceptos fundamentales

- modularización y abstracción
- tipos de datos: Primitivos y Objetos
- diagrama de clase
- creación de objetos
- diagrama de objetos
- sobrecarga de métodos
- interacción de objetos
- ocultando información
- Null
  - valores por defecto
- gestión interna de la memoria
  - tipos de datos primitivos
  - tipos de datos objeto
  - recolector de basura
  - tipos de datos primitivos – paso por valor
  - tipos de datos objeto – ¿paso por referencia?
- bibliotecas de Clases
- clase String
  - Identidad vs Igualdad
- documentación de clases
- uso de Javadoc para escribir documentación
- keyword this

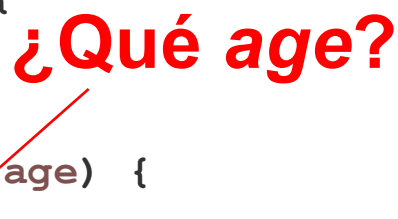
Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Keyword `this`

- Se usa para distinguir parámetros y campos con el mismo nombre

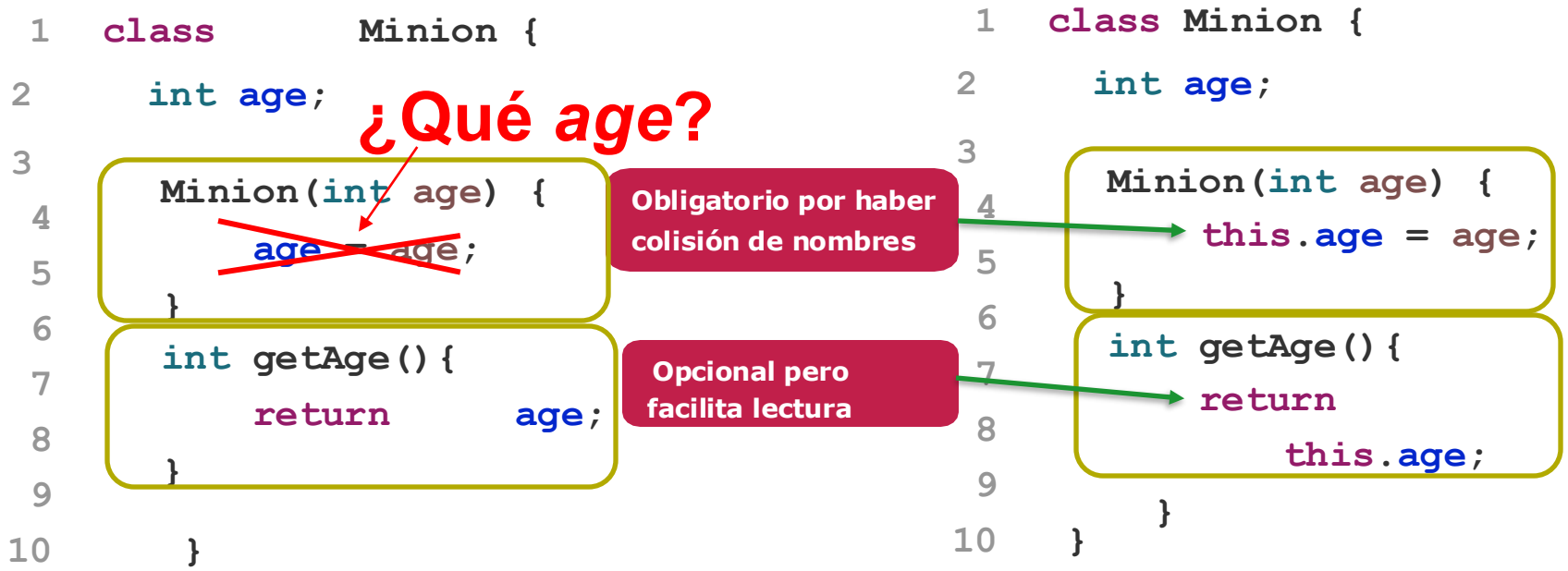
```
1   class Minion {
2       int age;
3
4       Minion(int age) {
5           age = age;
6       }
7       int getAge(){
8           return age;
9       }
10  }
```



Se realiza de forma implícita cuando no hay conflicto en los nombres

Keyword `this`

- Se usa para distinguir parámetros y campos con el mismo nombre



Se realiza de forma implícita cuando no hay conflicto en los nombres

Keyword this

- También se puede usar dentro de un constructor para invocar a un constructor sobrecargado de la misma clase

```
1  class Minion {
2      int age;
3      String name;
4      Minion(int age, String name) {
5          this.age = age;
6          this.name = name;
7      }
8      Minion(){
9          this(5743, "stuart")
10     }
11     Minion(int age){
12         this(age, "stuart")
13     }
14     Minion(String name){
15         this(5743, name)
16     }
17 }
```

Todos los atributos inicializados con valores a elegir

Todos los atributos inicializados con valores por defecto

Edad a elegir, nombre por defecto

edad por defecto, nombre a alegir

Si se usa, tiene que ser la primera sentencia en el constructor

Mejora la mantenibilidad

2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Ejercicios para pensar

Trata de resolverlos por tu cuenta

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Ejercicio 1

¿Cuál es la salida?

```
int a;
int b;
a=32;
b=a;
a=a+1;
System.out.println(b);
```

Usa el Bloc de Código (CodePad) de BlueJ para probar los ejercicios

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Ejercicio 2

¿Cuál es la salida?

```
NumberDisplay nd1 = new NumberDisplay(80);
NumberDisplay nd2 = nd1;
nd1.getValue()
nd2.getValue()
nd1.setValue(52);
nd1.getValue()
nd2.getValue()
nd2.increment();
nd2.getValue()
nd1.getValue()
```

No usamos ';' al final de sentencia solo en Bloc de Código (CodePad) de BlueJ para poder ver el valor devuelto por un método de acceso

Usa el Bloc de Código (CodePad) de BlueJ para probar los ejercicios

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

¿Qué es incorrecto?

```
/**
 * Print out info (number of entries).
 */
public void showStatus()
{
    if(files.size() == 0); {
        System.out.println("Organizer is empty");
    }
    else {
        System.out.print("Organizer holds ");
        System.out.println(files.size() + "files");
    }
}
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

¿Qué es incorrecto?

```
/**
 * Print out info (number of entries).
 */
public void showStatus()
{
    if(files.size() == 0); {
        System.out.println("Organizer is empty");
    }
    else {
        System.out.print("Organizer holds ");
        System.out.println(files.size() + "files");
    }
}
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

¿Y qué es incorrecto aquí?

Esta vez tenemos un campo booleano llamado 'isEmpty'...

```
/**
 * Print out info (number of entries).
 */
public void showStatus()
{
    if(isEmpty =  true) {
        System.out.println("Organizer is empty");
    }
    else {
        System.out.print("Organizer holds ");
        System.out.println(files.size() + "files");
    }
}
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

¿Y qué es incorrecto aquí?

Esta vez tenemos un campo booleano llamado 'isEmpty'...

```
/**
 * Print out info (number of entries).
 */
public void showStatus()
{
    if(isEmpty = true) {
        System.out.println("Organizer is empty");
    }
    else {
        System.out.print("Organizer holds ");
        System.out.println(files.size() + "files");
    }
}
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

La versión correcta

Esta vez tenemos un campo booleano llamado 'isEmpty'...

```
/**
 * Print out info (number of entries).
 */
public void showStatus()
{
    if(isEmpty == true) {
        System.out.println("Organizer is empty");
    }
    else {
        System.out.print("Organizer holds ");
        System.out.println(files.size() + "files");
    }
}
```

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Otra Bibliografía

- Tony Gauvin - Introduction to Programming. Presentations. University of Maine. Fort Kent.
- Christiane Hagedorn, Ann Katrin Kuessner, Thomas Staubitz, Ralf Teusner. Object-Oriented Programming in Java. OpenSAP Courses

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved

Otras fuentes de imágenes:

1. http://www.autoevolution.com/news/lexus-ct-200h-disassembled-after-100000km-german-endurance-test-79389.html
2. https://flylib.com/books/en/3.477.1.69/1/

Pearson

Copyright © 2017, 2012, 2009 Pearson Education, Inc. All Rights Reserved